Artificial Simulation of Neuronal Evolution

F. Bottega

October 25, 2026

Abstract

This article describes a computational model of digital organisms controlled by evolving artificial brains. Unlike traditional layer-based artificial neural networks, the proposed model represents each brain as a three-dimensional mesh of partially connected neurons, with topology, firing parameters, connectivity regions, and plasticity rules defined by a discrete genome. Selection is not defined by an explicit loss function, it emerges from survival and reproduction. We present the formalization of the genome, the rules of phenotypic generation, the signal propagation dynamics, the plasticity of connections, the recombination mechanism through offspring, and the implementation methodology in Godot. To allow lifetime learning without introducing a task-specific reward channel, the model is extended with a minimal endogenous modulatory field and synaptic eligibility traces whose organization is itself genetically determined.

Keywords: neuroevolution, genetic algorithms, artificial agents, neuronal plasticity, agent-based simulation, complex systems, endogenous modulation, meta-learning.

1 Introduction

In this work, each gene is a finite sequence composed of the numbers 0, 1, 2, 3 (representing the A, T, C, G in genetics). The notion of similarity between two genes is given by the L^1 distance, which is the absolute difference between corresponding digits in the sequence. For example, if $x = 12301$ and $y = 32101$, then

$$d(x, y) = |1 - 3| + |2 - 2| + |3 - 1| + |0 - 0| + |1 - 1| = 4.$$

Similar genes will produce similar brain features in our model. Although we will never compare genes of different lengths, we extend the definition of d to consider these cases. If the lengths of x and y are different, we append zeroes to the right of the shorter gene until they have the same length.

Each brain is a 3D tensor with dimensions $I, J, K \geq 2$. These dimensions are defined outside the simulations. This means we must test the evolution of many brain sizes in order to find the best ones. We will always consider[1] that $K \ll I, J$, although this is not mandatory.

[1]At the moment, we are working with moderate dimension sizes, therefore it is possible to load the entire brain with all of its parameters in the GPU at the same time. In the future this may not be feasible, when we try to scale the model. The motivation to consider $K \ll I, J$ is that we can work with only one slice of the form $[0, \ldots, I-1] \times [0, \ldots, J-1] \times k$ at a time in the GPU, for each $k \in \{0, \ldots, K-1\}$. This approach assures scalability to the model.

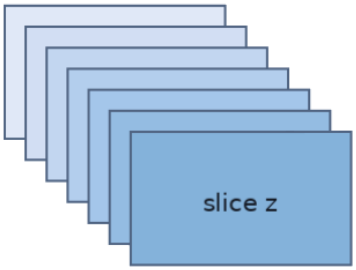


Figure 1: Structure of slices in the model.

The neuronal signals used in this model should not be interpreted as the amplitude of individual biological action potentials. Biological spikes are essentially all-or-none events, while information can be represented by their frequency and temporal distribution. Instead, the continuous signal produced by a neuron in our model may be interpreted as an aggregate measure of neural activity over a short temporal interval, analogous to averaging the firing activity of biological neurons within a given sampling window. Thus, a large signal corresponds to intense activity during that interval, whereas a small or zero signal corresponds to weak or absent activity. This abstraction preserves a meaningful relation with biological firing-rate dynamics without requiring the explicit simulation of individual spikes, which would substantially increase the computational cost of the model.

1.1 Support Functions

To model how a gene $x = x_0 \dots x_{p-1}$ generates a brain feature, we use functions of the form

$$\Phi(x) = \sum_{t=0}^{p-1} W(t) \cdot U(x_t), \tag{1}$$

where $W(t)$ are the *weights* of the function and $U(x_t)$ models how each digit of the gene acts on that feature. In this work, a few specific chosen W and U are used. However, in future work it is possible to test other families of functions or even allow these families to be genetically determined. What we need most from this model is for it to meet the following conditions:

1. The feature function is continuous over the real numbers.
2. Genes that are nearby (according to the L^1 metric) usually exhibit similar features.
3. In some cases, changing a single digit can have a significant impact.

The equations of this model can be combined with one another to obtain different distributions, depending on the objective at hand. Now we will discuss the specific functions chosen for our model.

Whenever we want a brain parameter ranging in some interval, we will be changing the length p of the gene and the base $\beta > 0$ in the equation

$$F_\beta(x) = \sum_{t=0}^{p-1} \sqrt{1+t} \cdot \beta^{x_t}. \tag{2}$$

Weighting the individual values by $\sqrt{1+t}$ is nothing special. The intention is to give positional weight to the gene digits. In particular, the positions of a gene are not required to

have the same phenotypic influence. Since $\sqrt{1+t}$ increases with t, later digits have a larger contribution to the scalar parameters defined through this family. This positional hierarchy is intentional and was verified numerically to provide useful variation without making the mapping excessively irregular.

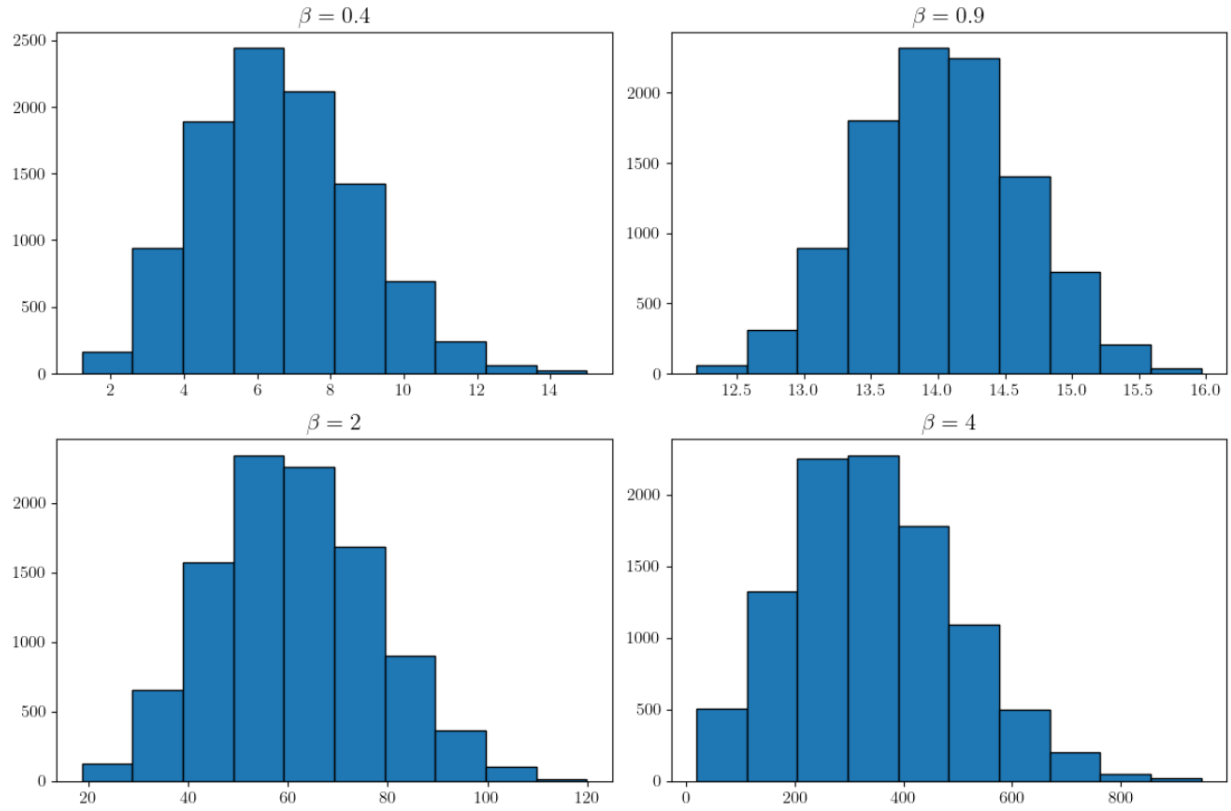


Figure 2: Distribution of F_β for $p = 8$ fixed and $\beta \in \{0.4,\ 0.9,\ 2,\ 4\}$. To generate the genes, each digit was drawn from the uniform distribution over $\{0, 1, 2, 3\}$.

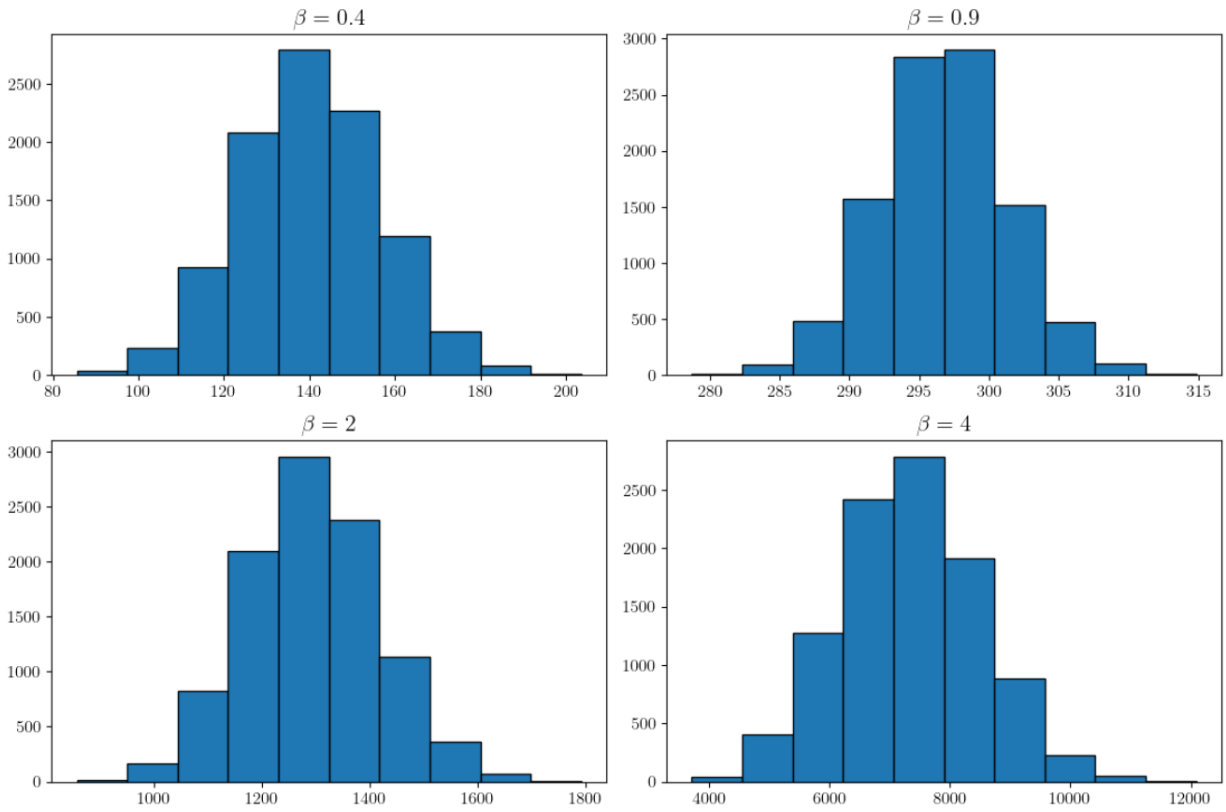


Figure 3: Distribution of F_β for $p = 64$ fixed and $\beta \in \{0.4,\ 0.9,\ 2,\ 4\}$. To generate the genes, each digit was drawn from the uniform distribution over $\{0, 1, 2, 3\}$.

We are also interested in knowing how nearby genes are mapped to nearby values by the defined mapping. For a fixed distance $\alpha \geq 0$ we have a level set $\{(x, y) : d(x, y) = \alpha\}$. We may regard this level set as the set of genes with the same degree of similarity. The relative difference $\frac{|F_\beta(x) - F_\beta(y)|}{F_\beta(x)}$ in this level set can vary, with this variance increasing with the distance. Below, we show the distributions of these differences for each level set.

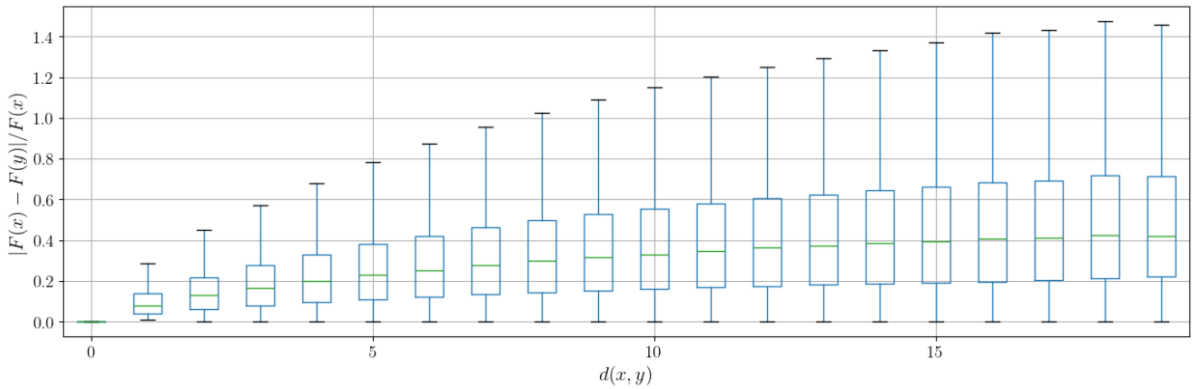


Figure 4: Box plot distribution of the relative differences $\frac{|F_\beta(x) - F_\beta(y)|}{F_\beta(x)}$ for $\beta = 0.4$ and 8 digits per gene. To generate the genes, each digit was drawn from the uniform distribution over $\{0, 1, 2, 3\}$.

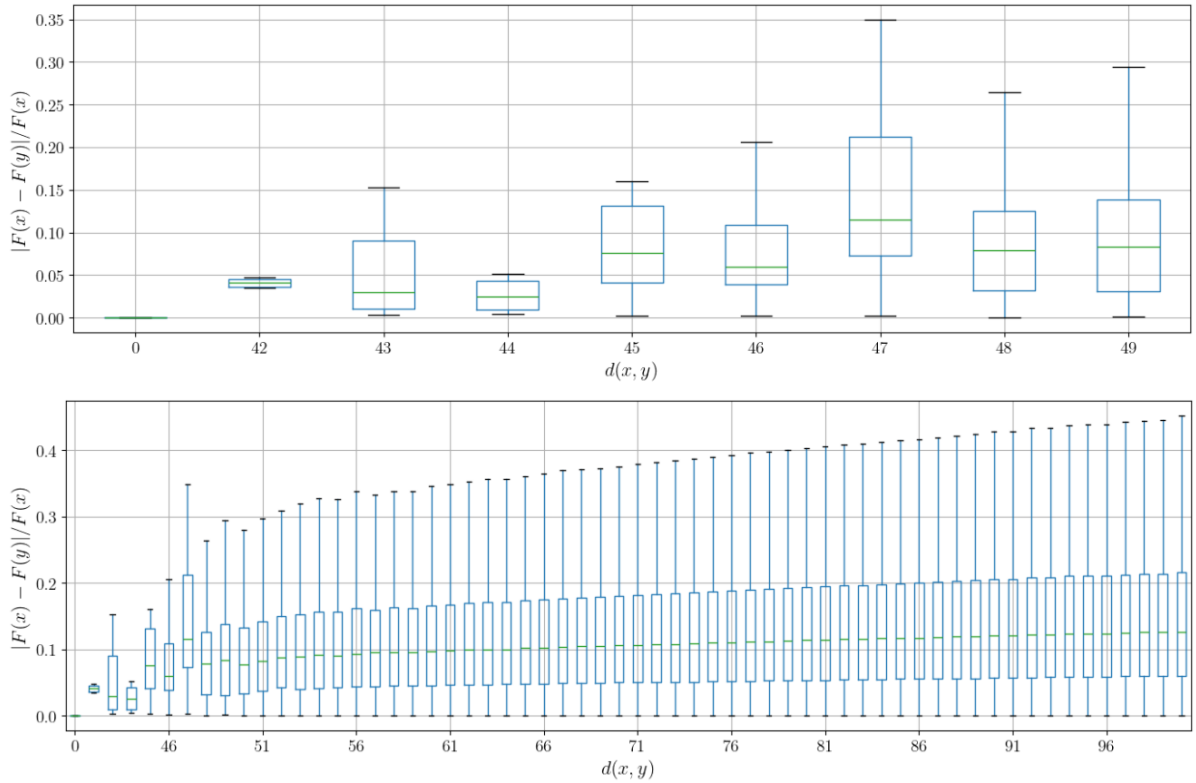


Figure 5: Box plot distribution of the relative differences $\frac{|F_\beta(x)-F_\beta(y)|}{F_\beta(x)}$ for $\beta = 0.4$ and 64 digits per gene. The first plot above is a zoomed-in view limited to $d(x, y) = 50$ for better visualization. The second plot covers the entire range obtained from the samples. To generate the genes, each digit was drawn from the uniform distribution over $\{0, 1, 2, 3\}$.

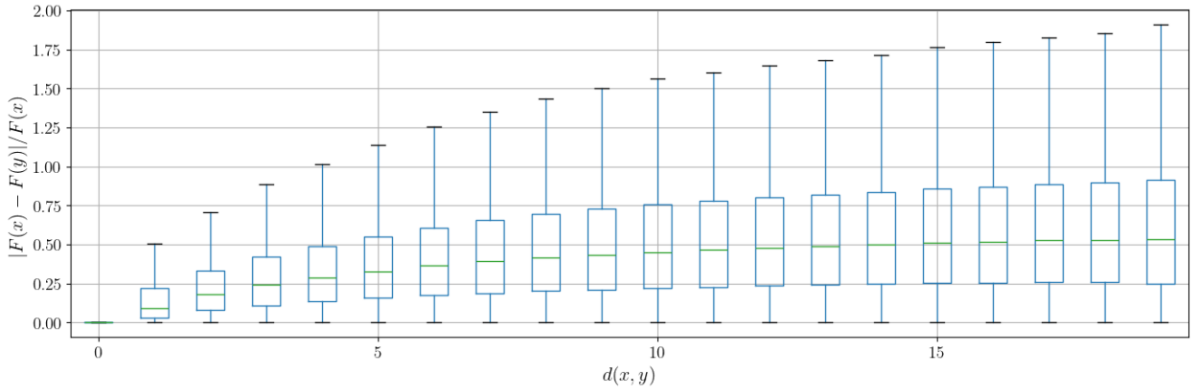


Figure 6: Box plot distribution of the relative differences $\frac{|F_\beta(x)-F_\beta(y)|}{F_\beta(x)}$ for $\beta = 4$ and 8 digits per gene. To generate the genes, each digit was drawn from the uniform distribution over $\{0, 1, 2, 3\}$.

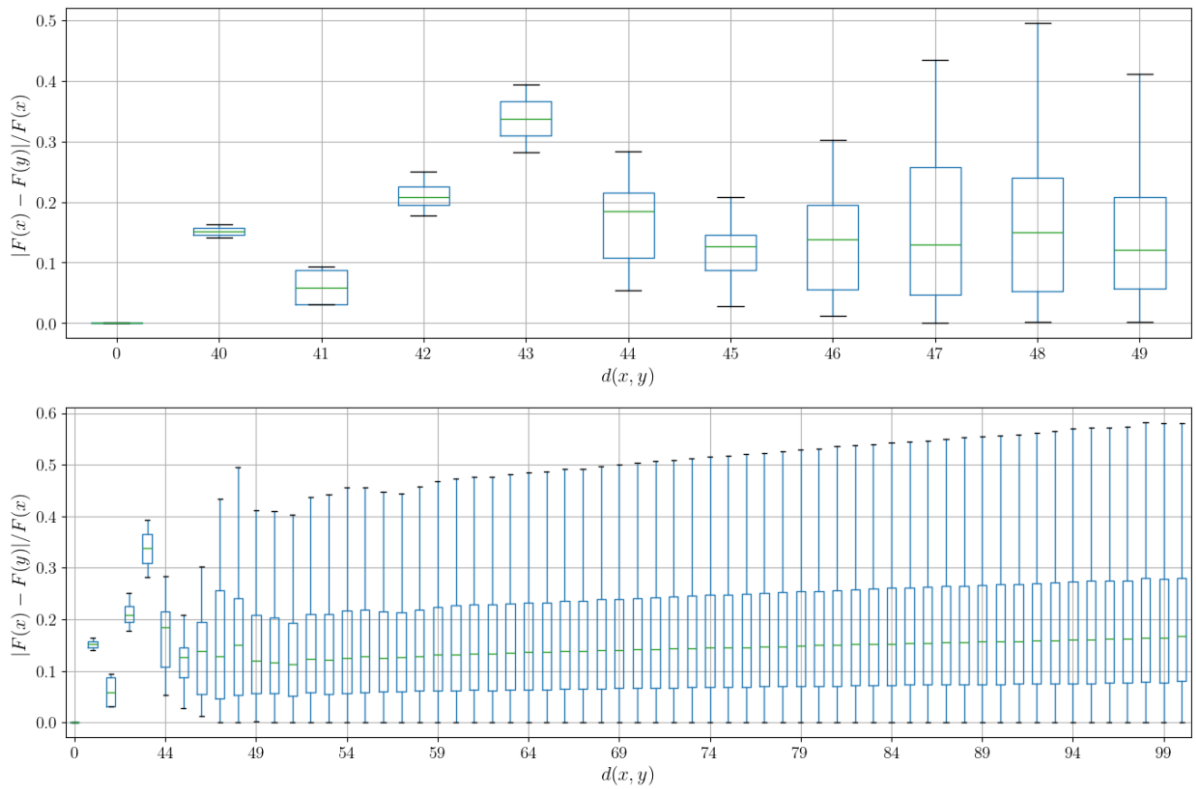


Figure 7: Box plot distribution of the relative differences $\frac{|F_\beta(x)-F_\beta(y)|}{F_\beta(x)}$ for $\beta = 4$ and 64 digits per gene. The first plot above is a zoomed-in view limited to $d(x, y) = 50$ for better visualization. The second plot covers the entire range obtained from the samples. To generate the genes, each digit was drawn from the uniform distribution over $\{0, 1, 2, 3\}$.

Although increasing p adds more terms to the sum, it also makes the total value more stable. The expected value of $F_\beta(x)$ grows approximately as $p^{3/2}$, while its typical fluctuations grow only as p. As a result, the relative variability scales roughly as $1/\sqrt{p}$. Therefore, larger values of p tend to produce smaller relative differences $|F_\beta(x) - F_\beta(y)|/F_\beta(x)$, even though the absolute differences may increase.

As explained, equation 2 may be regarded as a weighted sum. Another type of weighted sum is given by equation

$$G_{\delta_1,\delta_2}(x) = \sum_{t=0}^{p-1} \sin(1+t)\sin(1+x_t)^{\delta_1}\cos(1+x_t)^{\delta_2}. \tag{3}$$

The weights in this equation are the terms $\sin(1 + t)$. Furthermore, in this definition we have that $\delta_i \in \{0, 1\}$, with $\delta_1 + \delta_2 > 0$. Each choice gives a different kind of weighted sum.

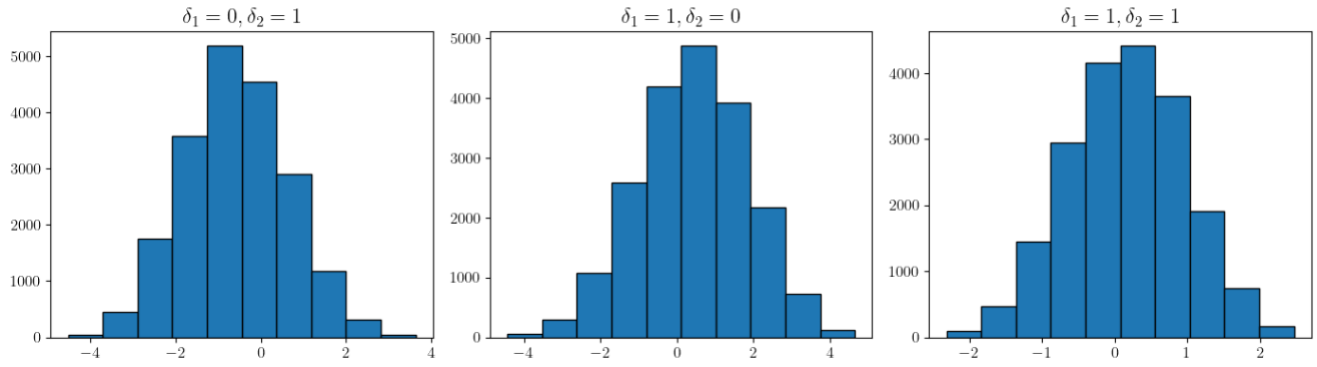


Figure 8: Distribution of G_{δ_1,δ_2} for 8 digits per gene. To generate the genes, each digit was drawn from the uniform distribution over $\{0, 1, 2, 3\}$.

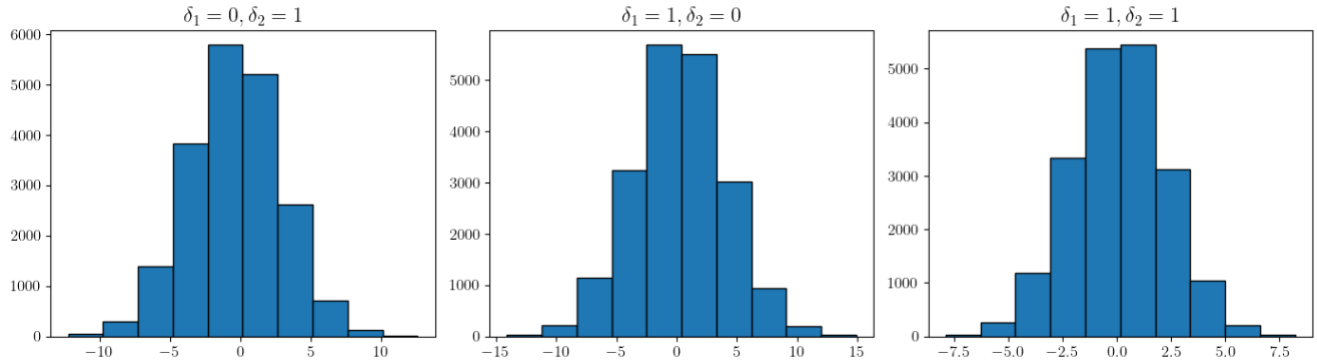


Figure 9: Distribution of G_{δ_1,δ_2} for 64 digits per gene. To generate the genes, each digit was drawn from the uniform distribution over $\{0, 1, 2, 3\}$.

Let (I, J, K) be the dimensions of the brain. The scalar field is defined over the normalized spatial coordinates

$$a = \frac{i}{I-1}, \qquad b = \frac{j}{J-1}, \qquad c = \frac{k}{K-1}.$$

Let the gene be $x = x_0 \dots x_{p-1}$ and assume that $p = n^3$. The correspondence between the gene index and the spatial mode is

$$t = q + ns + n^2 r.$$

Each digit of the gene is mapped to a scalar coefficient in $[-1, 1]$ according to

$$\alpha(x_t) = \frac{2x_t}{3} - 1.$$

Therefore,

$$0 \mapsto -1, \qquad 1 \mapsto -\frac{1}{3}, \qquad 2 \mapsto \frac{1}{3}, \qquad 3 \mapsto 1.$$

The scalar field is then defined by

$$A_{i,j,k}(x) = \tanh\left(\frac{1}{\sqrt{p}} \sum_{r=0}^{n-1} \sum_{s=0}^{n-1} \sum_{q=0}^{n-1} \alpha(x_{q+ns+n^2r}) \cos(\pi r a) \cos(\pi s b) \cos(\pi q c)\right). \tag{4}$$

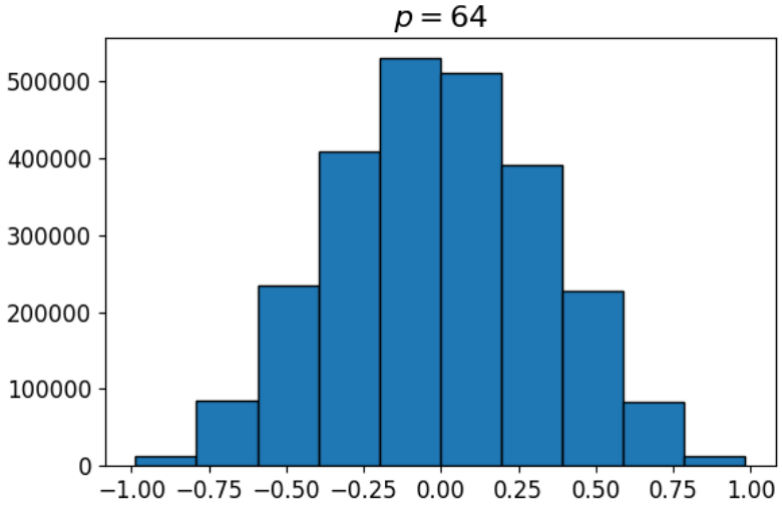


Figure 10: Distribution of $A_{i,j,k}(x)$ for $p = 64, I = 50, J = 50, K = 10$. To generate the genes, each digit was drawn from the uniform distribution over $\{0, 1, 2, 3\}$.

The factor $1/\sqrt{p}$ keeps the typical scale of the field approximately stable as the number of modes changes. Since the coefficients $\alpha(x_t)$ are centered around zero, the unnormalized sum exhibits cancellations and its typical magnitude grows on the order of $\sqrt{p}$ rather than p.

The correspondence $t = q + ns + n^2 r$ assigns each gene position to a unique spatial mode. Any mixing of spatial positions is therefore produced by the field itself rather than by an index collision. The positions of this gene are also not intended to be phenotypically equivalent. Here, however, the difference between positions is determined by the spatial mode (q, s, r) rather than by an explicit increasing weight. Low-frequency modes produce broad changes over the field, while higher-frequency modes produce finer spatial variation. In particular, $t = 0$ corresponds to the constant mode $(q, s, r) = (0, 0, 0)$ and therefore affects the field globally. Thus, single-digit changes can have different spatial effects depending on the mode encoded by that position. This is an intentional feature of the representation.

Another useful auxiliary is the following.

$$H_{i,j,k}(u, v, w, x) = \sin\left(\frac{3\pi}{8} + \frac{4}{C}\sum_{t=0}^{p-1}\sqrt{1+t}\cdot\Big(\sin\big(A_{i,j,k}(x)\cdot U(x_t)\big) + \cos\big(A_{u,v,w}(x)\cdot U(x_t)\big)\Big)\right), \tag{5}$$

where

$$C = \sum_{t=0}^{p-1}\sqrt{1+t}.$$

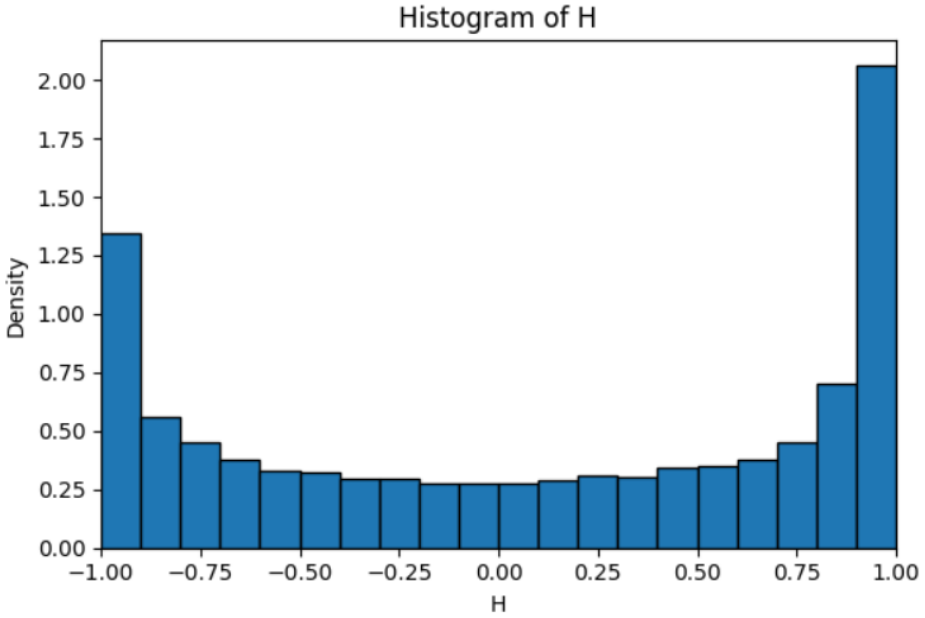


Figure 11: Distribution of $H_{i,j,k}(x)$ for $p = 64, I = 50, J = 50, K = 10$. To generate the genes, each digit was drawn from the uniform distribution over $\{0, 1, 2, 3\}$.

Unlike the definition of A, the same two values $A_{i,j,k}(x)$ and $A_{u,v,w}(x)$ are reused in every term of this sum. The normalization factor $4/C$, with C equal to the total positional weight, controls the scale of the weighted sum as the gene length changes. The outer shift is $3\pi/8$. The function H is used to determine which potential connections are eligible and to rank them during brain construction; its selected positive values also contribute to the initial synaptic weights through the source-dependent attenuation defined below.

This function is used when it is necessary to relate two distinct neurons through some value.

2 Description

We begin giving a list of items to describe our model of genetics and corresponding brain.

1. The maximum number N_{max} of outgoing connections assigned to any neuron during the initial generation of the brain is a global value of the brain. It is defined by the gene $x = x_0 \dots x_7$ through the formula

$$N_{max}(x) = \lfloor F_\beta(x) \rfloor = \left\lfloor \sum_{t=0}^{7} \sqrt{1+t} \cdot \beta^{x_t} \right\rfloor .$$

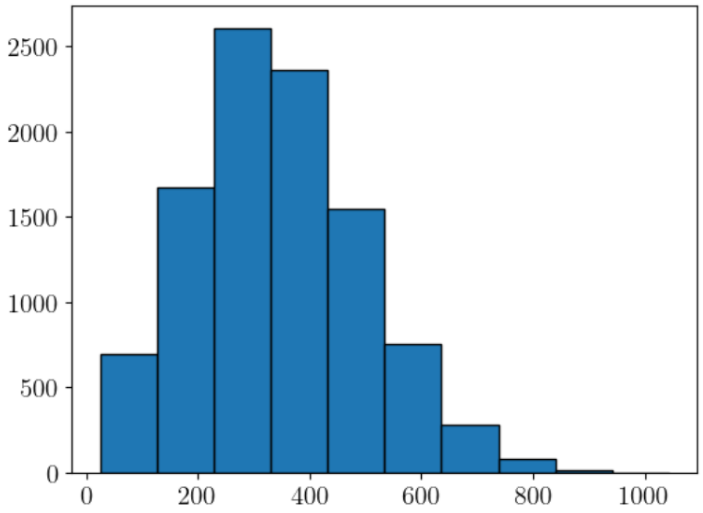


Figure 12: Distribution of N_{max} for $\beta = 4$. The horizontal axis shows the possible values of N_{max}, and the vertical axis gives their respective frequencies.

This definition of N_{max} depends on the value of β. Below, we show the highest value of N_{max} for a few choices of β.

β	$\max_x N_{max}(x)$
1	16
2	130
4	1043
8	8348

For $\beta \geq 1$, the highest value of N_{max} is approximately $16 \cdot \beta^3$. In practice, we want this maximum to be on the scale of IJ, meaning that the connection ceiling is comparable to approximately one entire slice. We therefore use $\sqrt[3]{IJ/16}$ as the reference scale for β.

The value of β is genetically determined. The gene $y = y_0 \dots y_7$ defines β through the formula

$$\beta(y) = 1 + \frac{\sum_{t=0}^{7} \sqrt{1+t} \cdot y_t}{3 \sum_{t=0}^{7} \sqrt{1+t}} \cdot \sqrt[3]{\frac{IJ}{16}}.$$

The fraction normalizes the weighted gene value to the interval $[0, 1]$. Therefore, the gene 00000000 produces $\beta = 1$, while 33333333 produces the maximum value $\beta_{\max} = 1 + \sqrt[3]{IJ/16}$. Intermediate genes produce intermediate values of β, with the contribution of each position weighted by $W(t) = \sqrt{1+t}$. The resulting ceiling may be somewhat larger than IJ at the upper end of the range, which is acceptable because N_{max} is only an upper bound on the number of connections assigned during the initial generation of the brain, not a prescribed number of connections. It does not impose a limit on connections created later by the lifetime dynamics. This makes the mapping from the gene to β monotonic and directly interpretable.

2. Let $n(i, j, k)$ denote the neuron at coordinate (i, j, k) in the brain. When the brain is being generated, this neuron may be connected to any other neuron $n(u, v, w)$ such that $u \in \{0, \ldots, I-1\}$, $v \in \{0, \ldots, J-1\}$, $w \in \{0, \ldots, K-1\}$, $|w-k| \leq 2$, and $(u, v, w) \neq (i, j, k)$.

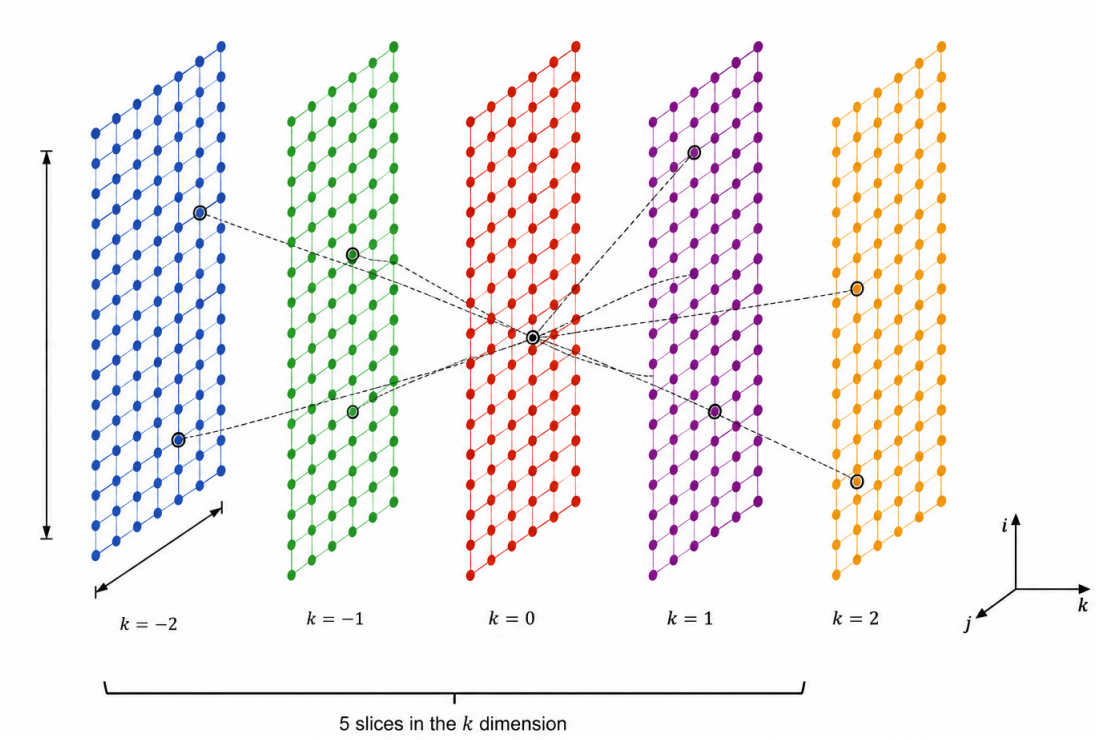


Figure 13: Region of possible connections for a single neuron.

The restriction to nearby slices is primarily computational rather than biological. The brain is intended to be processed slice by slice on the GPU, so limiting candidate targets to a small window of neighboring slices bounds the amount of structural data that must be simultaneously accessible. Within each eligible slice, however, the planar coordinates remain unrestricted. This preserves the possibility of long-range connections while keeping the memory-access pattern compatible with very large brains.

3. The gene that determines the initial connections is a sequence of the form $x = x_0 \ldots x_{63}$. For each neuron $n(i, j, k)$, a scalar activation field is defined over its three-dimensional neighborhood by

$$H_{i,j,k}(u, v, w, x).$$

For simplicity, we just call this gene the *connections gene*. In the definition, (u, v, w) are the coordinates of the neurons in the neighborhood of $n(i, j, k)$. We consider that $n(i, j, k)$ and $n(u, v, w)$ are connected if

$$H_{i,j,k}(u, v, w, x) > 0.$$

If more than N_{max} candidates satisfy this condition, they are ordered by their values of H and only the N_{max} largest values are retained. Thus, N_{max} acts as an upper bound on the outgoing degree during the initial generation of the brain rather than as a prescribed number of connections.

4. Each brain has a fixed number of neurons to receive inputs (external or internal signals supplied through the agent interface) and another fixed number of neurons to send outputs (actions on the environment or on the agent itself). Both of these numbers are determined outside the simulation. The input and output neurons form two distinct sets, that is, no neuron can be an input and output at the same time.

   Let $\mathcal{N}_{in}$ be the set of neuron inputs, $\mathcal{N}_{out}$ be the set of neuron outputs, and $\mathcal{N}_{int}$ be the set of interior neurons (not input and not output). To distribute input and output neurons more evenly across the brain, each slice k is ordered independently according to $A_{i,j,k}(x)$, with the connections gene x used in the procedure described above. Each value $A_{i,j,k}(x)$ is the *connection energy* of the neuron $n(i, j, k)$.

   Let

$$a_{k,1} \leq a_{k,2} \leq \cdots \leq a_{k,IJ}$$

   denote the neurons of slice k sorted by increasing connection energy. The final ordering is obtained by interleaving equal ranks from consecutive slices:

$$a_{0,1}, a_{1,1}, \ldots, a_{K-1,1}, a_{0,2}, a_{1,2}, \ldots, a_{K-1,2}, \ldots, a_{0,IJ}, \ldots, a_{K-1,IJ}.$$

   The first neurons in this ordering are assigned as inputs and the last ones as outputs. Thus, low and high energy neurons are selected across different slices before taking additional neurons from the same slice.

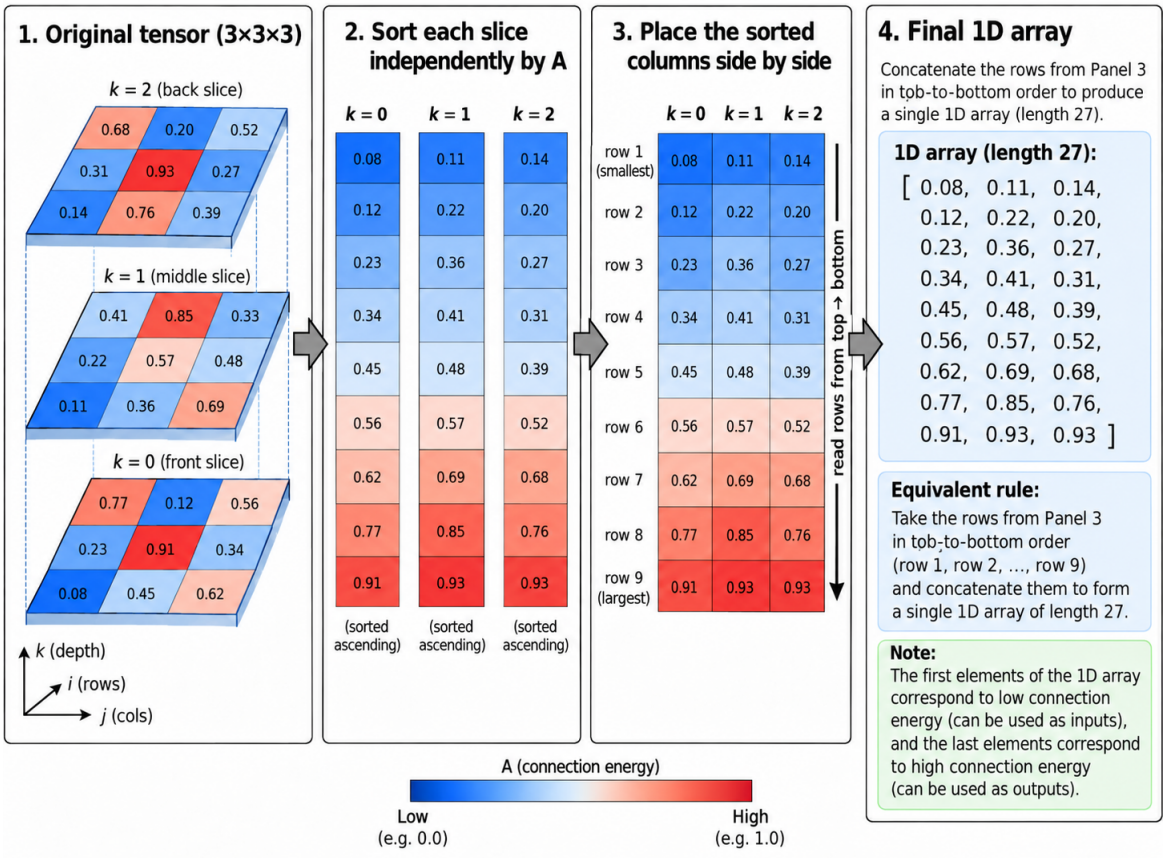


Figure 14: How the ordering works.

5. We require that input neurons send neural connections only to internal neurons and receive no neural connections from other neurons. Output neurons, conversely, receive

neural connections only from internal neurons and do not send neural connections to other neurons. Thus, inputs and outputs can interact with the neural network only through internal neurons, and no direct input–input, input–output, or output–output connection is allowed.[2] Therefore, a subroutine is executed to clear any invalid connections.

6. After the initial connections are created and invalid input/output connections are removed, some neurons may remain isolated with respect to the connectivity required by their role. Instead of immediately considering the brain invalid, a developmental rescue procedure is applied.

   Given a neuron n, define $\mathcal{I}(n)$ as being the number of incoming connections to n and $\mathcal{O}(n)$ as being the number of outgoing connections from n.

   - For every internal or output neuron $n(u, v, w)$ such that $\mathcal{I}(n(u, v, w)) = 0$, we search for neurons $n(i, j, k)$ that are allowed to connect to $n(u, v, w)$ according to the structural rules of the model, satisfy

     $$H_{i,j,k}(u, v, w, x) > 0,$$

     and still have available outgoing capacity under a slightly relaxed rescue limit,

     $$\mathcal{O}(n(i, j, k)) < \lceil 1.01 \cdot N_{\max} \rceil + 1.$$

     Among these candidates, the neuron producing the largest value of $H_{i,j,k}(u, v, w, x)$ is selected and a new connection to $n(u, v, w)$ is created. If the isolated target is internal, the source may be an internal or input neuron, provided that the connection is otherwise valid. If the isolated target is an output neuron, the source must be an internal neuron, since outputs receive neural connections only from internal neurons.
   - For every input neuron $n(i, j, k)$ such that $\mathcal{O}(n(i, j, k)) = 0$, the analogous procedure is applied in the outgoing direction. Only internal neurons $n(u, v, w)$ are considered valid targets. Among those satisfying $H_{i,j,k}(u, v, w, x) > 0$, the one producing the largest value of $H_{i,j,k}(u, v, w, x)$ is selected and a new connection is created. Input neurons are still subject to the same relaxed outgoing limit $\lceil 1.01 \cdot N_{\max} \rceil + 1$.

   The initial weight of every connection created by the rescue procedure is obtained from the same transformation used for the initial connections. Only one new connection is created for each neuron that requires rescue: one incoming connection for an isolated internal or output neuron, or one outgoing connection for an isolated input neuron. No existing connection is removed or redirected during this process. Therefore, if any such neuron cannot obtain a valid connection while respecting $H > 0$, the input/output restrictions, and $N_{\max}$, the construction is still considered a failure.

   This mechanism can be interpreted as a simplified developmental process in which poorly connected neurons promote the formation of a minimal additional synaptic contact while compatible neurons still retain the required structural capacity. It does not override the genetically defined connection field H, nor does it force a structurally saturated or otherwise incompatible brain to become viable. Consequently, sparse genomes are given an opportunity to produce valid brains without removing the selective pressure against genetically unfavorable connectivity patterns.

[2] Any input signal must be processed internally before reaching an output neuron.

7. Consider that all initial neural connections have been established, including any connections added by developmental rescue. In the following expression, x denotes specifically the *connections gene* used to compute H and the corresponding connection field A, not the gene associated with any other neuronal parameter. Let $w_{i,j,k}^{u,v,w} \in [0,1]$ denote the weight associated with a directional connection from $n(i,j,k)$ to $n(u,v,w)$. Its initial value, before enforcing the input-specific lower bound described later, is defined by

$$w_{i,j,k}^{u,v,w} = \left(1 - A_{i,j,k}(x)^2\right) H_{i,j,k}(u,v,w,x). \tag{6}$$

The attenuation factor depends only on the source neuron's value of the connection field A, and belongs to $[0,1]$. The positive values of H are used without attenuation to determine eligibility and to select the largest affinities, including during developmental rescue. Only after the complete connection topology has been determined is the factor $1-A_{i,j,k}(x)^2$ applied to each established connection. Consequently, this change affects the initial synaptic strengths but does not alter the connection rankings or the developmental connectivity. The same transformation applies to ordinary and rescued connections. For connections originating from input neurons, the input-specific minimum weight defined below is then enforced using the *input influence gene*, so their final initial weights may exceed the attenuated value in Equation 6.

8. Input neurons always accept external input signals and transmit them on. All the other neurons convert received signals into actions only under certain conditions[3]. Denote by $T(i,j,k) > 0$ the *activation threshold* of a neuron $n(i,j,k)$. Suppose that the state of the neuron is $s \geq 0$ (this value is the previous state of the neuron plus the sum of several signals coming from the connected neurons). Then the neuronal state is converted into an action only if

$$1 - e^{-s/\kappa} > T(i,j,k).$$

The function $\sigma(s) := 1 - e^{-s/\kappa}$ is called *saturation function* and it is used to normalize signals. Here $\kappa \geq 1$ is the *exponential factor* of the brain. It is a genetically defined parameter that controls how quickly a signal saturates. This value is determined by the formula

$$\kappa(x) = F_{0.4}(x) = \sum_{t=0}^{7} \sqrt{1+t} \cdot 0.4^{x_t},$$

where $x = x_0 \ldots x_7$ is the corresponding gene.

We don't need κ to assume high values, so the parameters in its formula are such that κ is approximately in the interval $[1,16]$. This is enough for our purposes.

Note that the saturation function is the same for the entire brain. What alters its behavior per neuron is the activation threshold of the neurons. These values are individual for each neuron. To motivate the definition of the threshold, first take a look at the saturation function σ below.

[3]In the case the neuron is not an output neuron this "action" is to send the signal to the other neurons. In the case the neuron is an output neuron, it will be actually converted into an action to be performed by the agent.

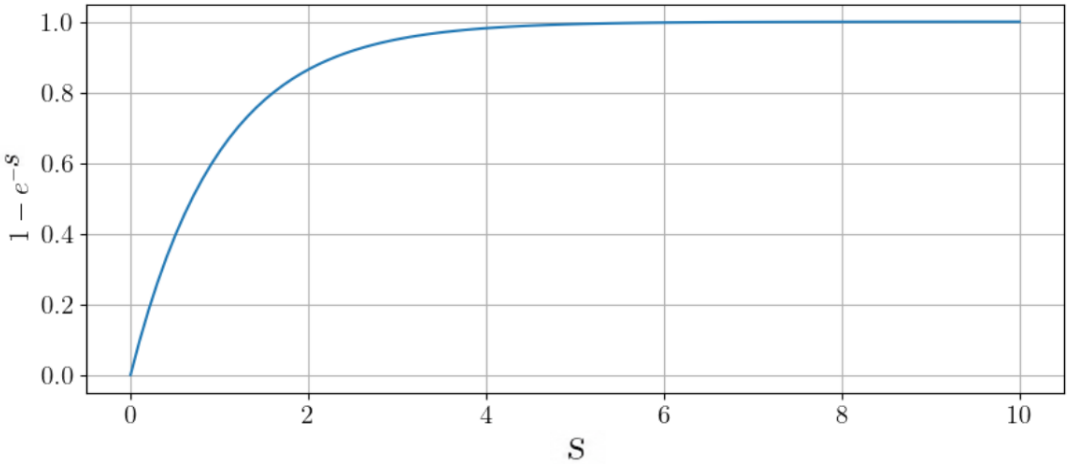


Figure 15: Saturation function $\sigma(s)$ for $s \in [0, 10]$.

In particular, note that

$$\sigma(0) = 0$$

and

$$\lim_{s \to +\infty} \sigma(s) = 1,$$

where the limit approaches from below. With this, we conclude that all T should be between 0 and 1. Now we are ready to define the formula for the activation threshold.

The gene that determines the activation threshold is a sequence of the form $x = x_0 \dots x_{63}$. Then for each neuron $n(i, j, k)$, the activation threshold function is defined by

$$T_x(i, j, k) = \frac{1 + A_{i,j,k}(x)}{2}.$$

Note that $T_x(i, j, k) \in (0, 1)$. The distribution of the values of this function is shown below.

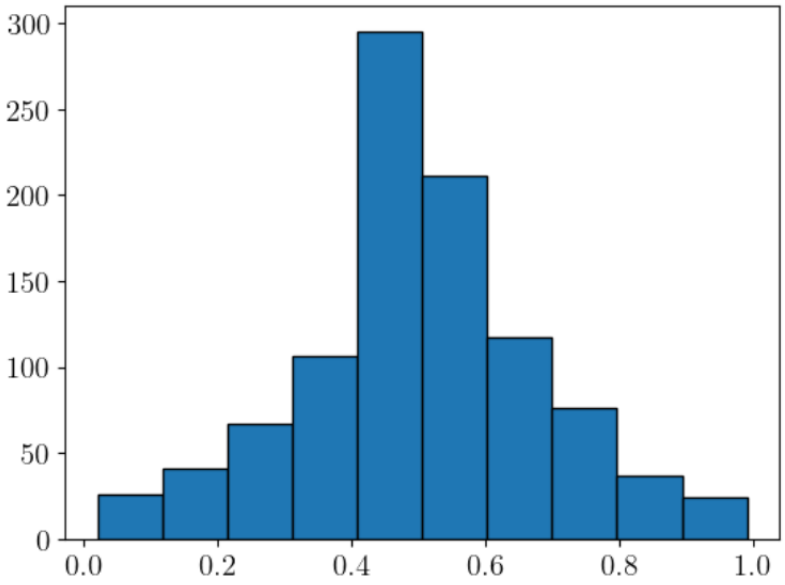


Figure 16: Distribution of the threshold values for an arbitrary neuron fixed and gene x variable. To generate the genes, each digit was drawn from the uniform distribution over $\{0, 1, 2, 3\}$.

9. If $n(i,j,k)$ is an input neuron, then $T_x(i,j,k) = 0$ by definition, although input neurons do not use the firing condition described above.

10. Whenever a neuron sends a signal, it enters a *refractory state*, meaning that it becomes more difficult to react to stimuli immediately afterward. This effect is represented by a non-negative variable ρ, called the *fatigue* of the neuron. Instead of using only $T_x(i,j,k)$ as the activation threshold, non-input neurons use the effective threshold

$$T_{\text{eff}}(i,j,k) = T_x(i,j,k)\,(1+\rho)\,.$$

To allow evolution to control the strength of this refractory response, the maximum fatigue is genetically determined. Let $x = x_0 \ldots x_7$ be the corresponding gene and define

$$\rho_{\max}(x) = F_{0.4}(x).$$

At each iteration, the current fatigue is used directly to determine the effective threshold, and the neuron is allowed to send a signal only if

$$\sigma(s_t(i,j,k)) > T_{\text{eff}}(i,j,k).$$

If the neuron sends a signal, its fatigue is reset to the genetically determined maximum value

$$\rho \leftarrow \rho_{\max}(x).$$

Otherwise, its fatigue is reduced according to

$$\rho \leftarrow \frac{\rho}{2}.$$

Thus, after a firing event, the maximum fatigue is used in the next firing test. In the absence of further firing, the refractory effect decreases geometrically across successive iterations. The order of these operations is illustrated in Figure 17.

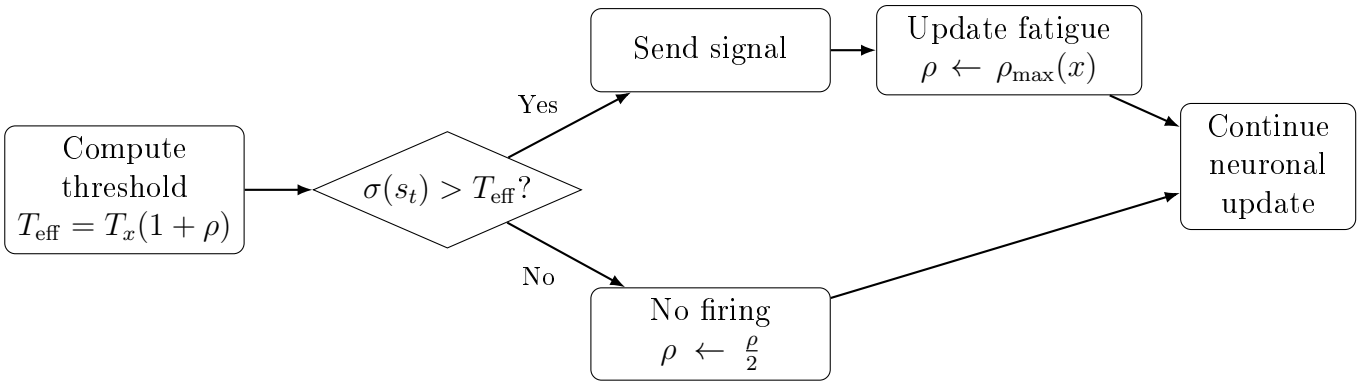


Figure 17: Order of the refractory-state update within one simulation iteration. The current fatigue determines the effective threshold. If the neuron fires, it sends a signal and its fatigue is reset to the genetically determined maximum value $\rho_{\max}(x) = F_{0.4}(x)$. Otherwise, the fatigue is reduced by half.

11. Input neurons do not have a refractory state.

12. If the neuron $n(i,j,k)$ sends a signal to $n(u,v,w)$, then the connection is reinforced. For a gene $x = x_0 \dots x_{63}$, define the *Hebbian plasticity rate*

$$\gamma_x(i,j,k) = \frac{1 + A_{i,j,k}(x)}{4}.$$

Note that $\gamma_x(i,j,k) \in [0, 1/2]$. Whenever the connection transmits a signal, its weight is updated by

$$w_{i,j,k}^{u,v,w} \leftarrow w_{i,j,k}^{u,v,w} + \gamma_x(i,j,k)\left(1 - w_{i,j,k}^{u,v,w}\right).$$

Values close to zero produce regions whose connections do not become strong merely because they are frequently used, while larger values preserve the fast associative reinforcement of the original model. This gives evolution control over the balance between stable and rapidly adapting regions.

13. The Hebbian rule above strengthens a connection because it was used, but it does not distinguish whether later activity of the organism makes that use worth preserving. We want to introduce this possibility without assuming that the environment sends a reward variable directly to the brain. The model below keeps only a small number of ingredients:

- neuronal activity may generate a local modulatory state
- this state may persist and spread through the brain
- different neurons may react differently to it
- recently used synapses may remain temporarily susceptible to change

No particular neuron type, brain region, reward meaning, or external teaching signal is prescribed in advance.

A gene $x = x_0 \dots x_{63}$ determines how strongly each neuron contributes to the modulatory state. Define the *modulatory release factor*

$$Q_x(i,j,k) = \frac{1 + A_{i,j,k}(x)}{2}.$$

Thus $Q_x(i,j,k) \in [0,1]$. Whenever $n(i,j,k)$ emits a signal at iteration t, it produces the local modulatory contribution

$$r_t(i,j,k) = Q_x(i,j,k)\,\sigma\left(s_t(i,j,k)\right),$$

and $r_t(i,j,k) = 0$ when the neuron does not emit a signal. This definition does not create a separate class of modulatory neurons. A neuron with $Q_x(i,j,k) \approx 0$ behaves, for this mechanism, as an ordinary neuron. A neuron with a large value can become a strong source. The same neuron may simultaneously participate in ordinary signal processing and in modulation.

Let $\mathcal{V}(i,j,k)$ be the set of valid orthogonally adjacent positions in the three-dimensional mesh,

$$\mathcal{V}(i,j,k) = \{(u,v,w) : |u-i| + |v-j| + |w-k| = 1\},$$

where only positions lying inside the mesh are retained. Thus, an interior neuron has at most six neighbors: $(i \pm 1, j, k)$, $(i, j \pm 1, k)$, and $(i, j, k \pm 1)$

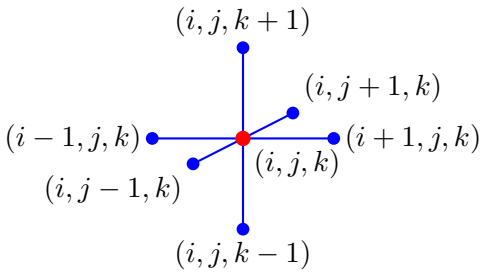


Figure 18: Illustration of the local neighborhood $\mathcal{V}(i, j, k)$ in the three-dimensional mesh. The red point denotes the central neuron, and the blue points are its six orthogonally adjacent valid neighbors.

A separate gene $z = z_0 \dots z_7$ determines three dynamical parameters,

$$\lambda(z) = \frac{1}{1 + e^{-G_{1,0}(z)}}, \qquad \nu(z) = \frac{1}{1 + e^{-G_{0,1}(z)}}, \qquad \mu(z) = \frac{1}{1 + e^{-G_{1,1}(z)}}.$$

The parameters λ, ν, and μ determine, respectively, the temporal persistence of the modulatory field, its spatial spread through the mesh, and the persistence of synaptic eligibility traces.

Let $c_t(i, j, k) \geq 0$ denote a scalar *modulatory field* at the position of the neuron. Starting from $c_0(i, j, k) = 0$, we update

$$c_t(i, j, k) = \lambda(z) \left[(1 - \nu(z)) c_{t-1}(i, j, k) + \frac{\nu(z)}{\operatorname{card}(\mathcal{V}(i, j, k))} \sum_{(u,v,w) \in \mathcal{V}(i,j,k)} c_{t-1}(u, v, w) \right] + r_t(i, j, k).$$

This is deliberately a general local field rather than a detailed simulation of a particular neurotransmitter. The retained properties are only local production, persistence, and spatial propagation. The average over $\mathcal{V}(i, j, k)$ is not an average of neuronal outputs, it is only a discrete local mixing rule that allows the field itself to propagate through the mesh. For small ν the influence remains concentrated near its sources, for large ν it spreads more broadly. Small λ produces brief events, while values close to 1 allow slowly varying modulatory states. Consequently, the same rule can produce almost isolated local effects, broad waves of modulation, persistent regional states, or useless saturated activity, depending on the genome and on the neuronal dynamics.

Another gene $y = y_0 \dots y_{63}$ determines how each neuron responds to the local field. Define the *modulatory sensitivity*

$$R_y(i, j, k) = A_{i,j,k}(y) \in [-1, 1].$$

The effective modulation at the neuron is

$$m_t(i, j, k) = R_y(i, j, k)\, \sigma_1 \left(c_t(i, j, k) \right).$$

The magnitude $|R_y|$ determines how sensitive that location is to the field, while its sign determines the direction in which the field influences synaptic plasticity. A value close to zero makes the region practically insensitive. This is intended only as an abstract counterpart of the fact that a released modulator has no fixed effect independently of the tissue that receives it. The response depends on the local biological machinery. No receptor classes or predefined positive and negative systems are introduced.

A modulatory event can occur after the synaptic activity that should be affected by it. Therefore, every existing directional connection stores an *eligibility trace*. Let

$$a_t(i,j,k,u,v,w) = \begin{cases} \sigma\left(s_t(i,j,k)\right)\sigma\left(s_t(u,v,w)\right), & \text{if the connection transmitted at } t, \\ 0, & \text{otherwise.} \end{cases}$$

The trace is updated by

$$e_t(i,j,k,u,v,w) = \mu(z)e_{t-1}(i,j,k,u,v,w) + (1-\mu(z))a_t(i,j,k,u,v,w), \qquad e_0 = 0.$$

Thus $e_t \in [0,1]$. The instantaneous eligibility is based on co-activation of the two ends of a connection, while the recurrence stores a fading memory of this local synaptic event. The parameter μ does not specify a fixed delay between action and consequence, it only determines how long the history of a used connection remains available. Evolution may therefore produce short or long temporal credit horizons.

After the ordinary Hebbian reinforcement above and before the decay of the connection, the endogenous correction is applied using the modulation at the postsynaptic neuron. To keep the weight in $[0,1]$ without introducing another learning-rate parameter, we use a symmetric bounded update:

$$w_{i,j,k}^{u,v,w} \leftarrow \begin{cases} w_{i,j,k}^{u,v,w} + m_t(u,v,w)e_t(i,j,k,u,v,w)\left(1 - w_{i,j,k}^{u,v,w}\right), & m_t(u,v,w) \geq 0, \\ w_{i,j,k}^{u,v,w} + m_t(u,v,w)e_t(i,j,k,u,v,w)w_{i,j,k}^{u,v,w}, & m_t(u,v,w) < 0. \end{cases}$$

Positive modulation moves an eligible connection toward 1, negative modulation moves it toward 0, and old or unrelated connections change little because their eligibility traces are small. The strength of the correction is not fixed by a separate global constant, it emerges from the amount of local field, the genetically defined sensitivity, and the recent use of the connection.

The purpose of this construction is not to force the evolution of a particular neuromodulatory architecture. There are no fixed modulatory regions, no source threshold, no receptor taxonomy, and no externally supplied scalar saying that an event was good or bad. A genome may produce a nearly ineffective modulatory system, destructive plasticity, diffuse noise, or highly organized local source/target patterns. The evolutionary process is responsible for preserving only the configurations that improve the ability of the organism to adapt during its lifetime.

14. At any iteration, the weight $w_{i,j,k}^{u,v,w}$ is reduced by a small factor given by the update formula

$$w_{i,j,k}^{u,v,w} \leftarrow w_{i,j,k}^{u,v,w} - e^{-12/D(i,j,k)}.$$

Here, $D(i,j,k) > 0$ is the *decay factor* associated with the neuron $n(i,j,k)$. This reduction is applied even in iterations where there is reinforcement.

The decay factor is defined by the gene $x = x_0 \ldots x_{63}$ through the formula

$$D(i,j,k,x) = 1.5 + A_{i,j,k}(x).$$

In particular, since $0.5 \leq D(i,j,k,x) \leq 2.5$, we always have

$$3.78 \times 10^{-11} \approx e^{-12/0.5} \leq e^{-12/D(i,j,k,x)} \leq e^{-12/2.5} \approx 8.23 \times 10^{-3}.$$

For a connection with initial weight 1 and no reinforcement, these bounds correspond to approximately 2.65×10^{10} iterations at the slowest-decaying extreme, $D = 0.5$, and approximately 122 iterations at the fastest-decaying extreme, $D = 2.5$. Assuming a reference execution rate of 60 iterations per second, these values correspond to approximate decay times of 14 years and 2.03 seconds, respectively.

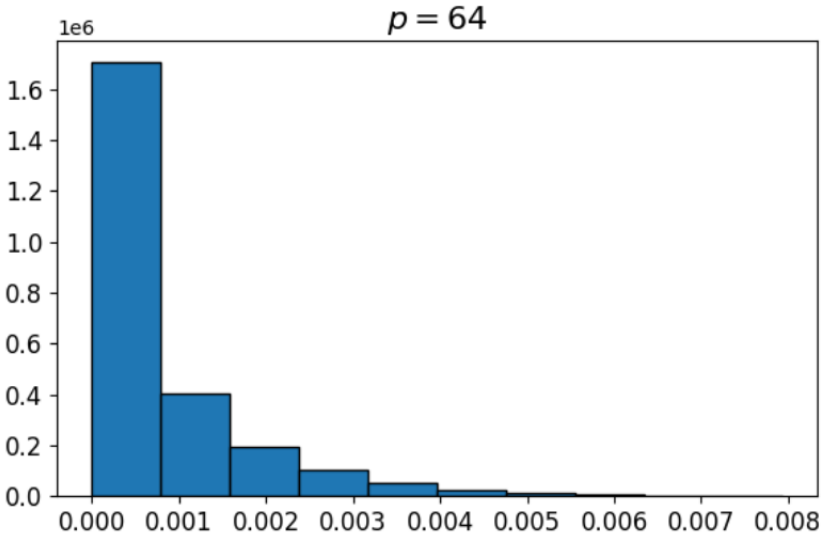


Figure 19: Distribution of $e^{-12/D(i,j,k,x)}$ for $p = 64, I = 50, J = 50, K = 10$. To generate the genes, each digit was drawn from the uniform distribution over $\{0, 1, 2, 3\}$.

If $w_{i,j,k}^{u,v,w} \leq 0$, the connection dies. To recreate the same connection again, it is necessary to go through the process described below.

15. The decay explained above does not apply to connections originating from input neurons.

16. Just to recapitulate, a non-input neuron $n(i, j, k)$ is able to emit a signal at time t if

$$\sigma(s) > T(i, j, k) \cdot (1 + \rho)\,.$$

Regardless of whether the neuron was able to emit the signal forward or not, its own state $s = s_t(i, j, k)$ is scaled over time by a factor $M \in [0, 1)$ called the *retention factor* of the neuron. At the end of the neuronal update, the state is updated by

$$s_t(i, j, k) \leftarrow M \cdot s_t(i, j, k).$$

The retention factor is defined by the gene $x = x_0 \ldots x_{63}$ through the formula

$$M(i, j, k, x) = \frac{1 - e^{-1-A_{i,j,k}(x)}}{1 - e^{-2}}.$$

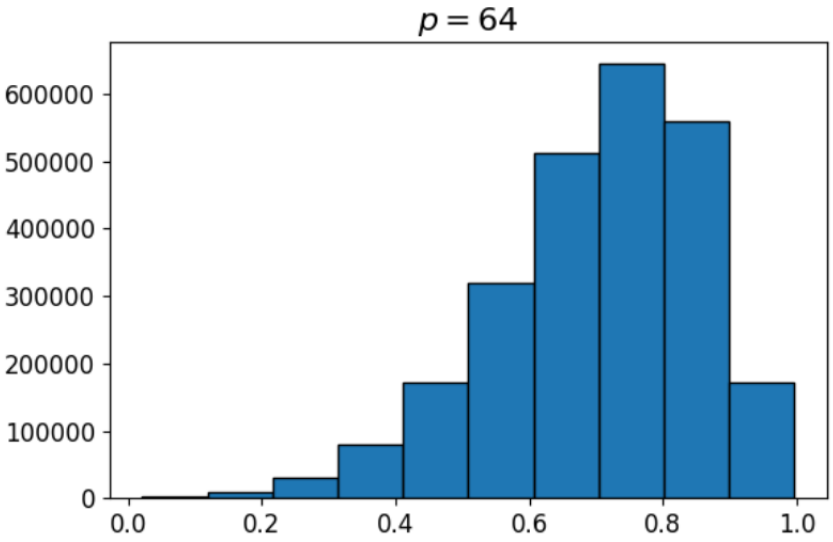


Figure 20: Distribution of $M(i,j,k,x)$ for $p = 64, I = 50, J = 50, K = 10$. To generate the genes, each digit was drawn from the uniform distribution over $\{0,1,2,3\}$.

17. For input neurons, the retention factor is set to $M = 0$ by definition. Therefore, an external input received in one iteration is transmitted during that iteration but is not retained as neuronal state afterward. A persistent sensory input must consequently be supplied again in each iteration in which the corresponding sensor remains active.

    This nonlinear mapping makes the retention factor increase smoothly with $A_{i,j,k}(x)$ for non-input neurons. Since the retention factor is applied at every iteration, it controls how quickly the residual neuronal state fades across successive updates.

18. Let $s = s_t(i,j,k)$ be the state of the neuron $n(i,j,k)$ at time t. Associated to each neuron there is a *polarity factor* $\alpha \in \{-1,1\}$ that defines the sign of the signal emitted by the neuron. Therefore, if the neuron $n(i,j,k)$ is allowed to emit its signal forward to $n(u,v,w)$, it will emit its weighed normalized signal with sign determined by its polarity factor. To be precise, it will emit the signal

$$\alpha \cdot w_{i,j,k}^{u,v,w} \cdot \sigma(s).$$

    If $\alpha = 1$, we say the neuron is *excitatory*, otherwise it is called *inhibitory*. The polarity factor is defined by the gene $x = x_0 \dots x_{63}$ through the formula

$$\alpha_x(i,j,k) = \begin{cases} -1, & A_{i,j,k}(x) < A_{0,0,0}(x), \\ 1, & A_{i,j,k}(x) \geq A_{0,0,0}(x). \end{cases}$$

    Since inhibitory signals can decrease the state of their targets, the sum of the incoming signals may cause a provisional neuronal state to become negative. However, neuronal states are restricted to $s \geq 0$ throughout the simulation. Therefore, whenever an update would produce a negative state, the lower bound is enforced by

$$s_t(i,j,k) \leftarrow \max\{0, s_t(i,j,k)\}.$$

    In this way, inhibition can completely suppress neuronal activity, but it cannot produce negative neuronal states.

19. Input neurons are always excitatory. Therefore, every connection originating from an input neuron uses $\alpha = 1$.

20. The gene that determines the threshold for the creation of new connections is a sequence of the form $x = x_0 \dots x_7$. This threshold is called *structural plasticity* and is defined by

$$P(x) = 1.75 + 0.25 \cdot \cos\left(G_{1,1}(x)\right).$$

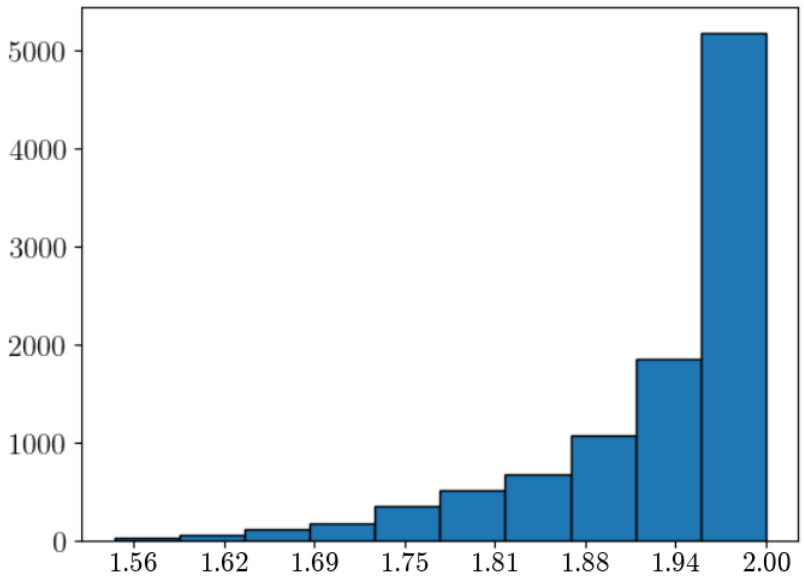


Figure 21: Distribution of the plasticity values. To generate the genes, each digit was drawn from the uniform distribution over $\{0, 1, 2, 3\}$.

In particular, note that $P(x) \in [1.5, 2]$. Connections involving input neurons do not participate in this procedure, since input neurons receive their activity directly from external signals and are therefore of a different nature.

To avoid storing an explicit history of neuronal signals, each non-input neuron $n(i, j, k)$ maintains a *activity trace* $R_t(i, j, k) \in [0, 1]$ representing its recent activity. The trace uses the same persistence parameter $\mu(z)$ previously introduced for the synaptic eligibility traces and is updated by

$$R_t(i, j, k) = \mu(z) R_{t-1}(i, j, k) + (1 - \mu(z))\, \sigma(s_t(i, j, k)), \qquad R_0(i, j, k) = 0.$$

Thus, R_t acts as a fading memory of the recent activity of the neuron.[4] The same genetically determined parameter $\mu(z)$ therefore controls the temporal persistence of both synaptic eligibility and neuronal activity traces. Small values of μ emphasize the most recent activity, whereas values close to 1 preserve information over a longer temporal interval. Since the update is a convex combination of values in $[0, 1]$, the activity trace also remains in $[0, 1]$. The trace is updated from the current normalized neuronal activity before structural plasticity is evaluated, so the test performed at iteration t uses R_t.

21. The neuronal activity trace is also used as an intrinsic homeostatic mechanism to prevent non-input neurons from remaining for long periods in extremely low or extremely high

[4] These values are stored regardless of whether the neuron has transmitted a signal during that iteration.

activity regimes. This mechanism acts only near the extremes of the activity range and does not interfere with neurons whose recent activity remains within the interval

$$L \leq R_t(i,j,k) \leq H,$$

where the fixed bounds are

$$L = 0.05, \qquad H = 0.95.$$

Since the saturation function is

$$\sigma(s) = 1 - e^{-s/\kappa},$$

the neuronal states corresponding to these two signal levels are

$$s_L = \sigma^{-1}(L) = -\kappa \ln(1-L), \qquad s_H = \sigma^{-1}(H) = -\kappa \ln(1-H).$$

For each non-input neuron, let s denote its current state and let R_t denote its current activity trace. The homeostatically corrected state is defined by

$$s \leftarrow \begin{cases} s + \dfrac{L - R_t}{L}(s_L - s), & R_t < L \text{ and } s < s_L, \\ s - \dfrac{R_t - H}{1 - H}(s - s_H), & R_t > H \text{ and } s > s_H, \\ s, & \text{otherwise.} \end{cases}$$

Thus, the mechanism is inactive for $L \leq R_t \leq H$. As R_t approaches either extreme, the correction increases continuously: low-activity states are moved toward s_L, while excessively high states are moved toward s_H. No additional genetically determined parameter is introduced, since the state scale follows directly from the exponential factor κ through the inverse saturation function.

22. At each iteration, if an internal neuron $n(i,j,k)$ fired a signal, then it is allowed to form one new outgoing connection. A neuron $n(u,v,w)$ is selected uniformly at random from the region of all possible connections. If the selected neuron is already connected to $n(i,j,k)$ or it is an input neuron, the process terminates without any connection being made. Otherwise, a new connection $w_{i,j,k}^{u,v,w}$ is created if

$$R_t(i,j,k) + R_t(u,v,w) > P(x).$$

Since both activity traces belong to $[0,1]$, their sum belongs to $[0,2]$, while $P(x) \in [1.5, 2]$. Consequently, the genetically determined plasticity threshold is restricted to the upper part of the possible combined-activity range, so a new connection can emerge only when the source and target have sufficiently high recent activity. Once the connection is created, its initial weight is

$$w_{i,j,k}^{u,v,w} = \frac{P(x)}{4},$$

so $w_{i,j,k}^{u,v,w} \in [0.375, 0.5]$. Unlike connections established during the initial development of the brain, these lifetime-generated connections do not use the connectivity function or its source-dependent attenuation to determine their initial weights. Here x continues to denote the gene defining P, not the connections gene.

Note that $n(u,v,w)$ may be an output neuron, so it is possible to form new connections between internal neurons and output neurons. However, it is not possible to form new connections between input neurons and internal neurons by this procedure.

23. Let r be the raw signal received by an input neuron. The model does not prescribe how raw external information must be encoded. Each input channel applies an externally defined mapping f that converts its raw input r, of arbitrary form, into a scalar $s = f(r) \in [0, 1]$. This result is the actual signal received by the neuron. These transformations are external to the brain and they depend entirely of the agent we want to build.

    The same goes to the outputs. After the final output is generated, it will be just a number. The subsequent action is a consequence of a function that determines what the agent should do based on the output number. These functions also are external to the brain.

    When a simulation runs to generated the brain evolution, the brain itself is the "being" we want to fit in the set of inputs/outputs transformations previously designed.

24. Merely allowing input neurons to transmit external signals forward is not sufficient, because their signals compete in the postsynaptic sum with signals arriving from other neurons. To give evolution control over how strongly input signals influence the internal brain dynamics, we introduce an *input influence gene*.

    Given a neuron n, define $\mathcal{I}(n)$ as the number of incoming connections to n and $\mathcal{O}(n)$ as the number of outgoing connections from n. Let $x = x_0 \dots x_7$ be the input influence gene. After the initial brain generation, we define the input influence factor

$$\psi(x) = \frac{1 + \sin\left(G_{1,1}(x)\right)}{|\mathcal{N}_{int}|} \sum_{n \in \mathcal{N}_{int}} \mathcal{I}(n).$$

    On average, an internal neuron has

$$\frac{1}{|\mathcal{N}_{int}|} \sum_{n \in \mathcal{N}_{int}} \mathcal{I}(n)$$

    incoming connections. This value is established during brain generation and remains fixed throughout the lifetime of the brain. Consequently, $\psi(x)$ is also fixed after generation and is not affected by connections subsequently created or removed by the lifetime dynamics.

    Let $s \in [0, 1]$ be the state of the input neuron $n(i, j, k)$ at a certain moment. This state is transmitted forward as the amplified signal $\psi(x)s$, allowing the input signal to compete with the aggregate activity received from other neurons. The normalization σ is not applied to the transmitted input signal, avoiding an unnecessary loss of information before this amplification. Since input neurons are excitatory by definition, the signal transmitted through an existing connection to an internal neuron $n(u, v, w)$ is

$$w_{i,j,k}^{u,v,w} \cdot \psi(x) \cdot s.$$

    The factor $1 + \sin\left(G_{1,1}(x)\right) \in [0, 2]$ controls how strongly the initial mean incoming degree contributes to the amplification of input signals. Its distribution is given below.

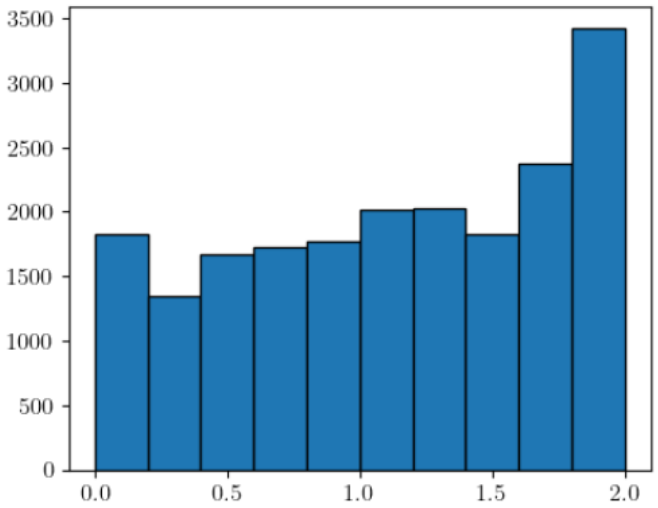


Figure 22: Distribution of the input influence factors $1 + \sin(G_{1,1}(x))$. To generate the genes, each digit was drawn from the uniform distribution over $\{0, 1, 2, 3\}$.

25. Connections originating from input neurons are preserved throughout the lifetime of the brain. They do not undergo passive synaptic decay, although Hebbian plasticity and endogenous modulation may still change their weights. To prevent these connections from becoming functionally negligible, their weights are constrained by

$$\frac{1}{1+\psi(x)} \leq w_{i,j,k}^{u,v,w} \leq 1.$$

After each synaptic update, this interval is enforced by

$$w_{i,j,k}^{u,v,w} \leftarrow \max\left\{ w_{i,j,k}^{u,v,w}, \frac{1}{1+\psi(x)} \right\}.$$

The lower bound depends on the genetically determined input influence itself. If $\psi(x) = 0$, the weight is forced to 1, but the transmitted input signal remains zero.

26. Connections from internal neurons to output neurons remain subject to passive synaptic decay, but the decay rate depends on the recent activity of the output neuron. For such connections, the decay step is

$$w_{i,j,k}^{u,v,w} \leftarrow w_{i,j,k}^{u,v,w} - R_t(u,v,w) \cdot e^{-12/D(i,j,k)}.$$

Since $R_t(u, v, w) \in [0, 1]$, connections terminating at outputs with high recent activity undergo decay at approximately the ordinary rate, whereas connections terminating at outputs that have remained inactive for a long period decay progressively more slowly.

As $R_t(u, v, w)$ approaches zero, passive decay may become arbitrarily small. Consequently, sufficiently strong connections associated with long-inactive outputs may persist indefinitely. This behavior is intentional: an output may represent a task or action that is not required for an extended period of the agent's lifetime, and previously established associations should not necessarily disappear solely because that behavior has remained unused. If the output becomes active again, its recent-activity trace increases and passive decay resumes accordingly.

27. If an output neuron remains inactive indefinitely after time t, then its recent-activity trace decays geometrically according to

$$R_{t+n}(u, v, w) = \mu(z)^n \cdot R_t(u, v, w).$$

Consequently, the total passive decay that an incoming connection can still accumulate is bounded by the geometric series

$$\sum_{n=0}^{\infty} R_{t+n}(u, v, w) e^{-12/D(i,j,k)} = \frac{R_t(u, v, w)}{1 - \mu(z)} e^{-12/D(i,j,k)}.$$

Therefore, if the current connection weight exceeds this finite remaining amount of decay, the connection will never reach zero through passive decay alone and may persist indefinitely.

28. The operations affecting a connection depend on the role of its source neuron. Input neurons transmit $w \cdot \psi \cdot s$ whenever an external signal is present. They bypass the ordinary activation threshold and refractory state, have retention factor $M = 0$, and do not maintain the activity trace R_t. Internal neurons transmit $\alpha \cdot w \cdot \sigma(s)$ only when they fire and follow the ordinary neuronal dynamics. Output neurons also follow the ordinary non-input dynamics, but they have no outgoing neural connections and expose their firing activity externally.

For input and internal sources, transmission is followed by Hebbian reinforcement when transmission occurred, then by the eligibility-trace update and endogenous modulation. The final step depends again on the source: input connections are clamped to the interval above and do not decay, while internal connections undergo passive decay and are removed if their weight becomes nonpositive.

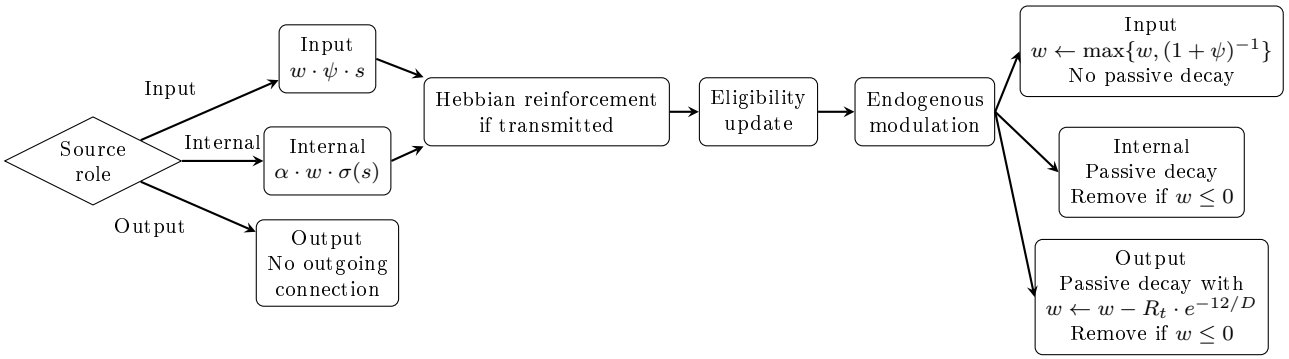


Figure 23: Temporal order of the operations affecting existing connections. Input and internal sources share the plasticity steps but differ in transmission and in the final weight treatment.

29. If the removal of a connection leaves an internal or output neuron $n(u, v, w)$ with no incoming connections, a homeostatic reconnection procedure is applied. Candidate sources are restricted to internal neurons $n(i, j, k)$ lying within the ordinary spatial connection region of the target, $|k - w| \leq 2$, with $(i, j, k) \neq (u, v, w)$.

Among all valid candidates, the source with the greatest recent activity trace is selected,

$$(i^*, j^*, k^*) \in \arg\max_{(i,j,k)} R_t(i, j, k).$$

If several candidates attain the same maximum value, one of them is selected uniformly at random. If no valid candidate exists, no replacement connection is created.

When a source is selected, a new directional connection from $n(i^*, j^*, k^*)$ to $n(u, v, w)$ is created with initial weight

$$w_{i^*,j^*,k^*}^{u,v,w} = \frac{P(x)}{4},$$

and its eligibility trace is initialized to zero.

This procedure does not preserve a synapse that has become ineffective. Instead, the old connection is allowed to disappear normally, while the disconnected neuron is given a new opportunity to receive activity from a recently active region of the brain. Unlike the genetically determined initial connectivity, this reconnection does not use H, its source is determined entirely by the recent dynamical state of the brain.

3 Overview

We have to choose the following parameters prior to the execution of the simulation:

- Dimensions I, J, K of the brain.
- Number of inputs $|\mathcal{N}_{in}|$ and number of outputs $|\mathcal{N}_{out}|$.

During the simulation, these are the genes that are meant to improve over time:

Gene feature	Gene size	Global parameter	Neuron parameter
Maximum number of initial connections	8	Yes	No
Base β for maximum initial connections	8	Yes	No
Initial connections	64	No	Yes
Activation threshold	64	No	Yes
Refractory strength	8	Yes	No
Exponential factor	8	Yes	No
Decay factor	64	No	Yes
Retention factor	64	No	Yes
Polarity factor	64	No	Yes
Hebbian plasticity rate	64	No	Yes
Modulatory release factor	64	No	Yes
Modulatory sensitivity	64	No	Yes
Modulatory dynamics	8	Yes	No
Structural plasticity	8	Yes	No
Input influence	8	Yes	No

Table 1: Genomic structure of the model, where each gene encodes a distinct functional parameter governing network dynamics.

In total, we have 568 digits making up the genome. This might not seem like much, but the total number of possible combinations is $4^{568} \approx 10^{342}$. When comparing this model to biological genomes, the raw scale difference is enormous. Real organisms range from millions to billions of base pairs. In a purely structural sense, this places it many orders of magnitude below even the simplest known biological genomes. However, this kind of comparison is not very informative, because biological genomes themselves contain large amounts of redundancy and non-coding or regulatory regions, and the relevant functional complexity is not proportional to raw size.

A more meaningful comparison is in terms of functional modules and degrees of freedom. In biology, complexity emerges from thousands of interacting genetic and regulatory components, while the model of this work compresses behavior into just 15 high-level genes that can encode rich, nonlinear transformations. This means the system is not attempting to replicate biological scale, but rather to explore how complex dynamics can arise from a highly compressed set of parameters. Its expressive power depends less on size and more on how much dynamical structure each gene can induce.

Organism	**Genome size**	**Gene count**
Model	5.68×10^2	15
Escherichia coli (K-12)	$\approx 4.6 \times 10^6$	$\approx$ 4,300
Drosophila melanogaster	$\approx 1.4 \times 10^8$	$\approx$ 13,600
Homo sapiens	$\approx 3.2 \times 10^9$	$\approx$ 20,000

Table 2: Comparison between a compressed genetic model and real biological genomes. Genome sizes refer to haploid assemblies. Gene counts are approximate functional gene annotations.

4 Endogenous Learning and Evolutionary Selection

The modulatory field above is part of the brain and not part of the environment. During the lifetime of an individual, the environment can influence learning only through quantities that reach the ordinary input neurons and through the physical consequences of the outputs. In particular, the synaptic update rule never receives a variable of the form

$$r_t = R(\text{environment state at } t)$$

whose purpose is to tell the brain how desirable the last event was. A change in energy, contact with an object, a visual pattern, or any other available quantity may still become important, but only if the ordinary neuronal dynamics transforms that information into internal activity that eventually affects the modulatory field.

This does not prevent the evolutionary simulation from selecting organisms by survival and reproduction. The distinction is between information available to selection across generations and information supplied directly to the synapses during one lifetime. Evolution may therefore favor genomes in which sensory and internal signals organize Q_x, R_y, the recurrent connectivity, and the eligibility dynamics into useful learning mechanisms, without the simulator specifying what those mechanisms should regard as reward.

The construction was intentionally kept continuous and non-categorical. A neuron is not declared to be modulatory, it merely has a genetically determined value of Q_x. A region is not declared to be a receptor region, it merely has a genetically determined value of R_y. The field is not assigned a biological identity, it is only a local state that can be produced, persist, spread, and alter eligible synapses. Specialized source populations, local or broad neuromodulation, insensitive regions, opposing responses, and different temporal credit horizons are therefore possible configurations of the same small set of rules rather than separate mechanisms inserted by hand. Bad configurations are equally possible and are expected to be eliminated only by evolution.

For the long-term objective of learning to learn, the evolutionary protocol is as important as the neuronal equations. If all generations experience essentially the same situation, selection can encode a useful behavior directly in the genome. A stronger test is to evolve under heterogeneous conditions and then freeze a selected genome and place the resulting brain in environments

that were not used during selection. Improvement during the lifetime of the same individual, especially when it disappears after synaptic plasticity is disabled, is evidence that evolution produced a learner rather than only a fixed behavioral solution.

5 Inputs and Outputs

The model treats inputs and outputs as general interfaces between the brain and the agent, rather than restricting them to external sensory signals and motor actions. Inputs may encode internal variables such as hunger, energy, global or regional neural activity, arousal, or other observable physiological states. Likewise, outputs may represent not only actions on the external environment, but also actions on the agent itself, including modulation of internal physiological variables or direct stimulation of selected brain regions. This allows the evolved brain to potentially learn both environmental behavior and self-regulation without relying on an externally provided reward signal.

Because neural and physiological states can themselves become inputs, and outputs can act on those states or directly stimulate brain regions, the model also admits closed internal feedback loops in which brain activity can influence its own future activity even when external stimuli do not change. Together with recurrent connectivity and persistent neuronal states, such loops may permit self-generated neural dynamics from which processes such as recall, planning, or internally generated thought could emerge. Whether richer phenomena associated with consciousness can arise from these mechanisms is left as an open question rather than assumed by the model.

6 Offspring

Let $X = (x^{(1)}, \ldots, x^{(n)})$ and $Y = (y^{(1)}, \ldots, y^{(n)})$ be two individuals (genomes), each composed of n genes[5]. The offspring generation process can be described in the following steps.

1. The crossover step is only applied to homologous genes. In our setting, that would be genes with the same superscript.

2. Given two homologous genes $x^{(i)}$ and $y^{(i)}$, the procedure makes a cut every 4 positions of the gene. For example, if $x^{(i)} = x_0^{(i)} \ldots x_7^{(i)}$ and $y^{(i)} = y_0^{(i)} \ldots y_7^{(i)}$, then the cut separates each gene into two blocks:

$$x_0^{(i)} x_1^{(i)} x_2^{(i)} x_3^{(i)} \mid x_4^{(i)} x_5^{(i)} x_6^{(i)} x_7^{(i)}$$

$$y_0^{(i)} y_1^{(i)} y_2^{(i)} y_3^{(i)} \mid y_4^{(i)} y_5^{(i)} y_6^{(i)} y_7^{(i)}$$

   The corresponding gene of the child will be assembled block by block, randomly selecting one of the parents' blocks to place in the appropriate position. Let $z^{(i)}$ be the child's corresponding gene. Then there are 4 possibilities:

$$z^{(i)} = x_0^{(i)} x_1^{(i)} x_2^{(i)} x_3^{(i)} \mid x_4^{(i)} x_5^{(i)} x_6^{(i)} x_7^{(i)}$$

$$z^{(i)} = x_0^{(i)} x_1^{(i)} x_2^{(i)} x_3^{(i)} \mid y_4^{(i)} y_5^{(i)} y_6^{(i)} y_7^{(i)}$$

$$z^{(i)} = y_0^{(i)} y_1^{(i)} y_2^{(i)} y_3^{(i)} \mid x_4^{(i)} x_5^{(i)} x_6^{(i)} x_7^{(i)}$$

[5] Currently we are working with the $n = 15$ genes shown in Table 1, but this can change in the future.

$$z^{(i)} = y_0^{(i)} y_1^{(i)} y_2^{(i)} y_3^{(i)} \mid y_4^{(i)} y_5^{(i)} y_6^{(i)} y_7^{(i)}$$

The random sampling is 50/50, there is no preference for any parent. Also, note that this procedure extends naturally for any gene with length multiple of 4, which is the case of our model (we only work with genes of length 8 or 64).

3. After all crossovers are finished, there is a final step of mutation. All digits of the genome are subject to mutation with probability $p_{mut} = 0.001$ (0.1%), where each selected digit is replaced by a randomly sampled value from its allowed range. This value is defined outside the simulation and can be changed.

7 Memory Consumption

At each iteration, the neurons of a single slice $[0, \dots I-1] \times [0, \dots J-1] \times k$ are updated. These slices are used to store the parameters of the neurons. More precisely, each entry (i, j) is responsible for storing 4 parameters of the neuron (i, j, k), using the pattern RGBA. In fact, we will need 3 of these matrices $I \times J$ per slice, which gives a total of 12 parameters per neuron. This doesn't account for parameters regarding connections.

The learning and structural-plasticity mechanisms add three kinds of persistent dynamic state. The modulatory field requires one scalar $c_t(i, j, k)$ per neuron. Since its update uses values from the previous iteration, an implementation can alternate two single-channel buffers for c_{t-1} and c_t. The recent-activity mechanism requires one scalar $R_t(i, j, k)$ per non-input neuron, because each trace depends only on its own previous value and the current neuronal activity, it can be updated in place. The eligibility mechanism requires one scalar $e_t(i, j, k, u, v, w)$ per existing connection. The values Q_x, R_y, λ, ν, and μ are determined by the genome and coordinates and therefore do not need to be stored as independent mutable state. The fixed input influence factor ψ requires only one additional scalar for the entire brain and therefore has negligible memory cost.

The field update and the activity-trace update are local to the neuronal representation and do not require storing an explicit history of previous signals. Their additional memory is therefore linear in the number of neurons, while the eligibility memory is linear in the number of available connection slots. If a single-channel texture format is used, the two field buffers require $2IJKp$ bytes and the activity traces require at most a further $IJKp$ bytes, where p is the number of bytes per stored scalar.

Since N_{max} limits only the initial developmental connectivity, it cannot also be used as the lifetime storage limit. For a fixed-capacity GPU representation, let N_{slot} denote the number of connection slots reserved per neuron. This is an implementation capacity rather than a genetically determined model parameter and may be chosen larger than N_{max} or increased when necessary. Such an implementation may store neuronal connections in a 3D tensor of shape $I \times J \times N_{\text{slot}}$. Each entry (i, j, k') gives the data about the k'-th connection associated to the neuron $n(i, j, k)$. If the k'-th connection does not exist yet, we can just flag it using the value RGBA $= (-1, -1, -1, -1)$. Otherwise, there will be a tuple of the form (u, v, w, ϕ), where u, v, w are the coordinates of the target neuron and ϕ is some float.[6] The eligibility trace may be stored in an additional single-channel buffer of size $IJKN_{\text{slot}}$, or packed into unused connection-data channels when available. The table below shows the options of types to use.

[6] At the moment, we are encoding the indexes with float numbers. The `float16` type is able to represent integers up to 2048 appropriately and this is enough for our purposes for now. If bigger indexes are necessary, one may use `float32` or use only integers and consider the float as a sequence of bits representing it.

Format	Components	Type	Bytes	Range	Example
FORMAT_R8	R	uint8 normalized	1	$[0, 1]$	(0.75)
FORMAT_RG8	RG	uint8 normalized	2	$[0, 1]$	$(0.20, 0.80)$
FORMAT_RGBA8	$RGBA$	uint8 normalized	4	$[0, 1]$	$(0.1, 0.5, 0.8, 1.0)$
FORMAT_RH	R	float16	2	Float	(-3.5)
FORMAT_RGH	RG	float16	4	Float	$(2.0, -1.2)$
FORMAT_RGBAH	$RGBA$	float16	8	Float	$(1.0, -2.0, 0.5, 7.3)$
FORMAT_RF	R	float32	4	Float	(-123.456)
FORMAT_RGF	RG	float32	8	Float	$(1.5, -4.2)$
FORMAT_RGBAF	$RGBA$	float32	16	Float	$(1.0, -2.0, 0.5, 100.0)$

Table 3: Main image formats available in Godot for 2D and 3D textures.

The suffixes `H` and `F` indicate the numerical type used to store each texture component. The suffix `H` (Half Float) corresponds to a 16-bit floating-point number, occupying 2 bytes per component, while the suffix `F` (Float) corresponds to a standard 32-bit floating-point number, occupying 4 bytes per component. Consequently, a texture in the `RGBAH` format stores four half-precision floating-point values (8 bytes per texel[7]), whereas `RGBAF` stores four single-precision floating-point values (16 bytes per texel).

Half-precision (`float16`) provides approximately 3 to 4 significant decimal digits, whereas single-precision (`float32`) provides approximately 7 significant decimal digits. It is important to note that these values refer to *significant digits* rather than digits after the decimal point. Consequently, the absolute precision depends on the magnitude of the represented value: numbers close to zero are represented with finer resolution than large-magnitude numbers.

[7] A *texel* (texture element) is the basic storage unit of a texture, analogous to a pixel in an image. Each texel stores one or more numerical components depending on the texture format (e.g., R, RG, or $RGBA$), and each component may be represented as an integer or floating-point value. In the context of this work, texels are used as generic data containers rather than colors, storing the variables required by the simulation.

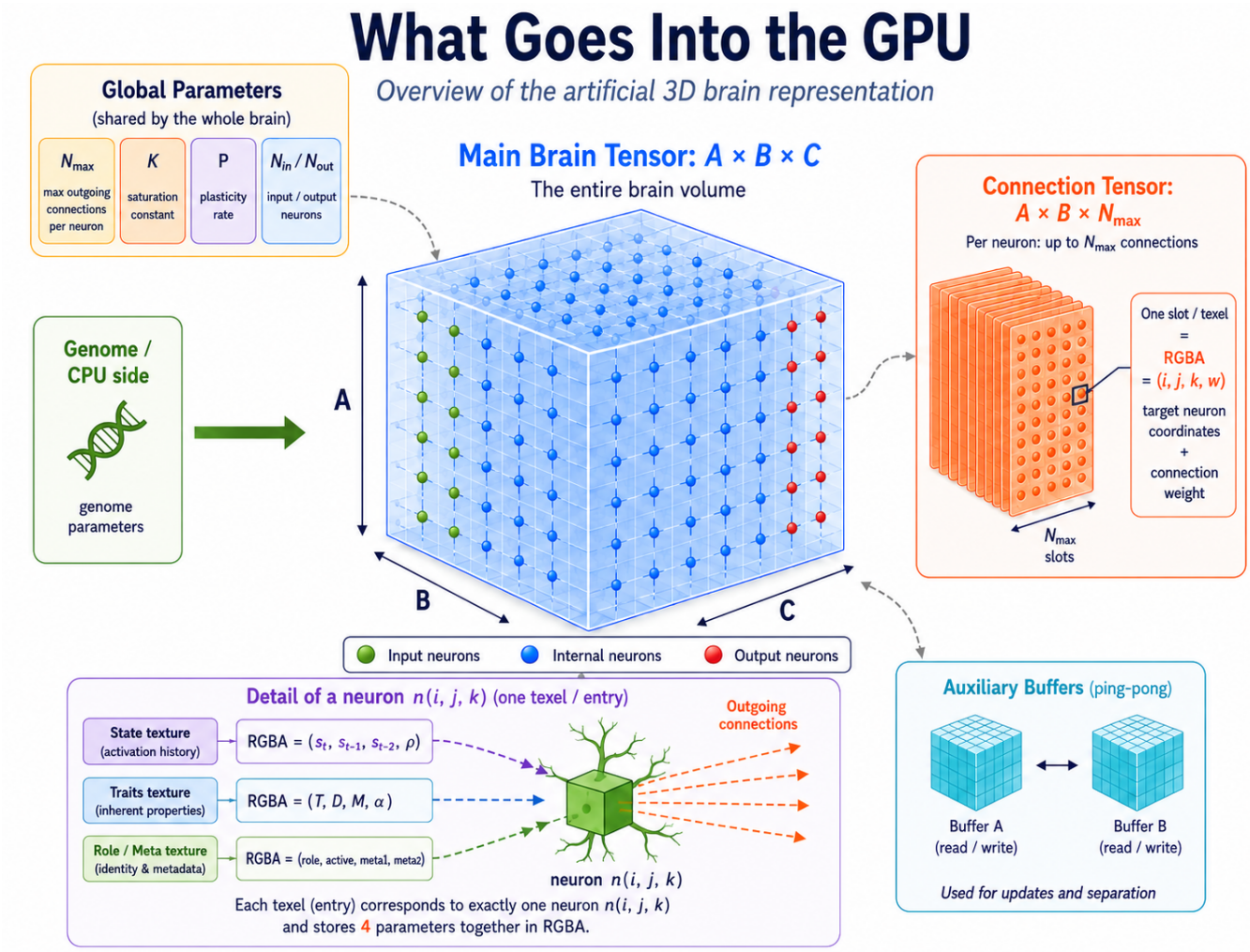


Figure 24: Overview of the brain structure and how its parameters are represented in the GPU.

7.1 Estimates

To determine the GPU memory footprint for the simulation model, we define the total storage required based on the structural tensor configurations. Let I and J represent spatial dimensions, K the number of slices, and N_{slot} the connection-slot capacity used by the implementation.

The estimates below describe the baseline neuron and connection tensors. The two single-channel modulatory-field buffers discussed above add $2IJKp$ bytes, and the neuronal activity traces add at most a further $IJKp$ bytes. If eligibility traces are stored in a separate single-channel buffer rather than packed into an existing connection tensor, they add a further $IJKN_{\text{slot}}p$ bytes.

Each slice contains three baseline $I \times J$ matrices and a certain number of auxiliary connection tensors of size $I \times J \times N_{\text{slot}}$ (we show in the table below the memory cost of using up to three of these auxiliary connection tensors). Let m be the number of the connection tensors per slice. Since each grid entry stores 4 floating-point components (RGBA), the total logical elements E_{slice} per slice is

$$E_{slice} = (I \cdot J \cdot 3) + (m \cdot I \cdot J \cdot N_{\text{slot}}) = I \cdot J \cdot (3 + m \cdot N_{\text{slot}})$$

The total memory consumption Mem (in bytes) accounts for the total number of elements across all K slices, multiplied by the 4 components per element and the precision byte-size p:

$$Mem = K \cdot E_{slice} \cdot 4 \cdot p,$$

where p is determined by the data type (`float16`: $p = 2$ bytes, `float32`: $p = 4$ bytes). The final value in Gigabytes (GB) is obtained by dividing Mem by 1024^3.

8 Compute Shaders

The original implementation strategy described above represents neuronal and connection data through RGBA textures. This organization is convenient because fragment shaders naturally process textures in parallel, with each texel storing several numerical parameters. However, the brain model is not fundamentally an image-processing problem. Its main computational objects are neurons and directional connections, both of which must be updated repeatedly and, in the case of connections, may also be created, removed, or modified during the lifetime of the agent.

For this reason, compute shaders provide a more natural implementation model. Instead of associating one execution with one rendered fragment, a compute shader launches a large number of independent GPU invocations over arbitrary data buffers. The simulation can therefore be organized directly around the mathematical objects of the model.

For example, one dispatch may assign one GPU invocation to each neuron:

```
index = global_invocation_id

if index < number_of_neurons:
 read neuron[index]
 update state
 update fatigue
 update modulatory variables
 write neuron[index]
```

A different dispatch may assign one invocation to each available connection slot:

```
index = global_invocation_id

if index < number_of_connection_slots:
 read connection[index]

 if connection exists:
  update eligibility trace
  update synaptic weight
  apply decay
 end
end
```

This organization avoids forcing the simulation into the geometry of a rendered image. Neuronal data may still be packed in groups of four floating-point values when convenient, similarly to RGBA, but these groups can simply be stored as vectors inside linear GPU buffers. For example, a neuron with linear index

$$n = i + Ij + IJk$$

may be accessed directly as

```
neuron = neurons[n]
```

while, in a fixed-slot implementation, the c-th connection associated with this neuron may be stored at

$$q = nN_{\text{slot}} + c,$$

and accessed as

```
connection = connections[q]
```

The same linear indexing can be used both in the CPU reference implementation and in the GPU implementation.

The main advantage of this approach becomes more important as the brain grows. A model containing millions of neurons may contain tens or hundreds of millions of active connection slots. Compute shaders allow these large collections to be processed with massive GPU parallelism while preserving a direct correspondence between one invocation and one simulated object. In contrast, a conventional fragment-shader implementation would require the simulation to be expressed through rendering passes, textures, and rasterization constraints that are unrelated to the underlying neural dynamics.

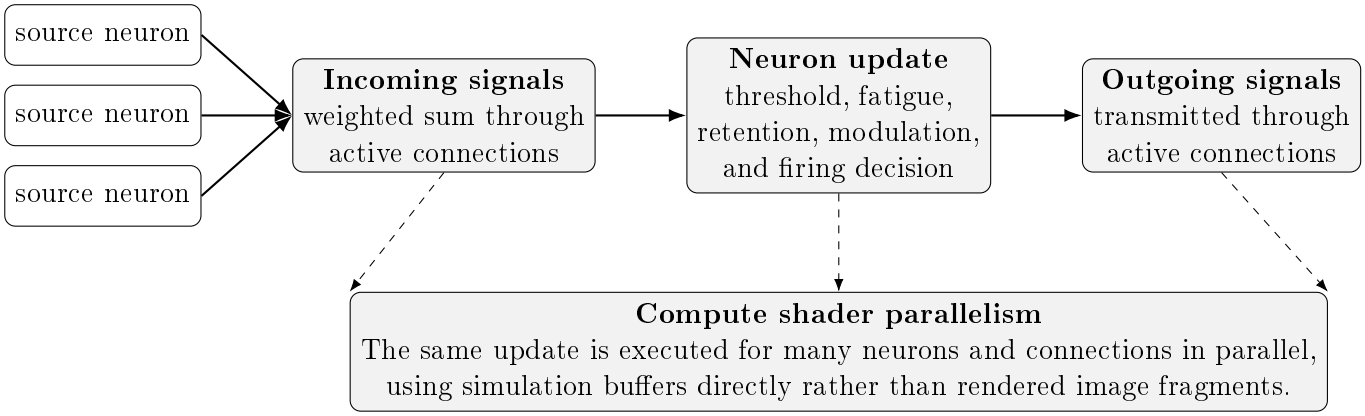


Figure 25: Schematic view of the neuronal dynamics. Each neuron receives weighted signals, updates its internal state, and may then transmit signals through its outgoing connections. Compute shaders are well suited to this model because the same operations can be applied in parallel to large numbers of neurons and connections.

Compute shaders are also particularly suitable for the structural plasticity of the model. Connection slots can be treated as mutable data entries rather than texels whose interpretation is constrained by the rendering pipeline. This makes operations such as changing weights, invalidating connections, updating eligibility traces, or inserting new connections considerably more direct.

The intended architecture is therefore to keep high-level simulation control, visualization, file handling, and interaction in GDScript, while transferring the large parallel numerical updates to compute shaders. A simplified simulation step may be organized as follows:

1. `Determine the current normalized neuronal activities, update the neuronal activity traces, firing states, refractory updates, and modulatory releases.`

2. Update the modulatory field and compute the effective modulation.

3. Process every existing directional connection once:
   a. If the source fires, accumulate the transmitted signal using the pre-update connection weight.
   b. Apply ordinary Hebbian reinforcement when transmission occurs.
   c. Update the synaptic eligibility trace.
   d. Apply postsynaptic endogenous modulation.
   e. Apply synaptic decay and remove the connection if its weight becomes nonpositive.

4. Apply homeostatic reconnection to targets that lost their last incoming connection.

5. Advance neuronal states to the next iteration using retention and the accumulated incoming signals.

6. For each internal neuron that fired, evaluate at most one structural plasticity attempt using the neuronal activity traces.

The connection-processing stage preserves the temporal order defined by the model while allowing transmission, Hebbian reinforcement, eligibility, endogenous modulation, and decay to be performed during a single traversal of the existing connections. Some stages may still require separate GPU dispatches to guarantee that quantities computed for iteration t, such as the firing states and modulatory field, are available to all connections before subsequent updates begin. When necessary, two buffers may be alternated in a ping-pong scheme so that one contains the previous state while the other receives the new state.

				GPU Memory Consumption (GB)					
				float16 (2 bytes/value)			**float32** (4 bytes/value)		
I	**J**	**K**	$\mathbf{N_{slot}}$	**1 Tensor**	**2 Tensors**	**3 Tensors**	**1 Tensor**	**2 Tensors**	**3 Tensors**
128	128	16	16	0.04	0.07	0.10	0.07	0.14	0.20
128	128	16	128	0.26	0.51	0.76	0.51	1.01	1.51
128	128	16	1024	2.01	4.01	6.01	4.01	8.01	12.01
128	128	128	16	0.30	0.55	0.80	0.59	1.09	1.59
128	128	128	128	2.05	4.05	6.05	4.09	8.09	12.09
128	128	128	1024	16.05	32.05	48.05	32.09	64.09	96.09
256	256	16	16	0.15	0.27	0.40	0.30	0.55	0.80
256	256	16	128	1.02	2.02	3.02	2.05	4.05	6.05
256	256	16	1024	8.02	16.02	24.02	16.05	32.05	48.05
256	256	128	16	1.19	2.19	3.19	2.38	4.38	6.38
256	256	128	128	8.19	16.19	24.19	16.38	32.38	48.38
256	256	128	1024	64.19	128.19	192.19	128.38	256.38	384.38
512	512	16	16	0.59	1.09	1.59	1.19	2.19	3.19
512	512	16	128	4.09	8.09	12.09	8.19	16.19	24.19
512	512	16	1024	32.09	64.09	96.09	64.19	128.19	192.19
512	512	128	16	4.75	8.75	12.75	9.50	17.50	25.50
512	512	128	128	32.75	64.75	96.75	65.50	129.50	193.50
512	512	128	1024	256.75	512.75	768.75	513.50	1025.50	1537.50
1024	1024	16	16	2.38	4.38	6.38	4.75	8.75	12.75
1024	1024	16	128	16.38	32.38	48.38	32.75	64.75	96.75
1024	1024	16	1024	128.38	256.38	384.38	256.75	512.75	768.75
1024	1024	128	16	19.00	35.00	51.00	38.00	70.00	102.00
1024	1024	128	128	131.00	259.00	387.00	262.00	518.00	774.00
1024	1024	128	1024	1027.00	2051.00	3075.00	2054.00	4102.00	6150.00
2048	2048	16	16	9.50	17.50	25.50	19.00	35.00	51.00
2048	2048	16	128	65.50	129.50	193.50	131.00	259.00	387.00
2048	2048	16	1024	513.50	1025.50	1537.50	1027.00	2051.00	3075.00
2048	2048	128	16	76.00	140.00	204.00	152.00	280.00	408.00
2048	2048	128	128	524.00	1036.00	1548.00	1048.00	2072.00	3096.00
2048	2048	128	1024	4108.00	8204.00	12300.00	8216.00	16408.00	24600.00

Table 4: Comparative GPU memory allocation analysis for an artificial brain simulation model across three structural scaling scenarios. Parameters are derived from a framework where each logical simulation layer contains K independent slices. Basic neuron characteristics are stored via three static grid matrices of size $I \times J$ per slice, because each entry holds 4 floating-point numbers (RGBA channels), this accounts for exactly 12 baseline structural parameters per neuron. Synaptic connectivity and weights are modeled via 3D tensors of dimension $I \times J \times N_{\text{slot}}$, where each RGBA entry encodes connection pathways. The memory footprint scales across three execution configurations requiring either one (1 Tensor), two (2 Tensors), or three (3 Tensors) auxiliary connection maps per slice. The total logical element count per configuration is mathematically given by $K \times I \times J \times (3 + m \cdot N_{\text{slot}})$, where $m \in \{1, 2, 3\}$. Memory statistics are represented in binary gigabytes (GiB, denoted as GB) to depict static VRAM usage on the device for `float16` (2 bytes per value) and `float32` (4 bytes per value) precisions.